

CUSTOMER RETENTION INTELLIGENCE SYSTEM

Phases:

Phase 1 — Python Core (CP1–CP5)

Phase 2 — Database Layer (SQL1–SQL6 + ORM)

Phase 3 — FastAPI Backend (API1–API6)

Phase 4 — React UI (RE1–RE5)

Phase 5 — Data Engineering (DE1–DE8, incl. PySpark)

Phase 6 — Machine Learning (ML1–ML5)

By:

Yuvasree R

82586

PHASE 1 - PYTHON CORE

LAB CP1 — Profile the Customer Dataset

This section involved exploring and profiling the customer churn dataset by examining its structure, data types, missing values, categorical feature distributions, churn rate, and key numerical statistics. Data quality checks were also performed on the TotalCharges column, and the findings were summarized to support data preprocessing and model development.

1. Load customer_churn.csv using pandas and print its shape (rows, columns)

```
import pandas as pd
df = pd.read_csv("customer_churn.csv")
print("=" * 50)
print("Dataset Shape")
print("=" * 50)
print(f"Rows: {df.shape[0]}")
print(f"Columns: {df.shape[1]}")
```

```
=====
Dataset Shape
=====
```

```
Rows: 7043
Columns: 21
```

2. Print all column names and their data types

```
print("\n" + "=" * 50)
print("Column Names and Data Types")
print("=" * 50)
print(df.dtypes)
```

```
=====
Column Names and Data Types
=====
```

```
customerID      str
gender          str
SeniorCitizen   int64
Partner         str
Dependents      str
tenure          int64
PhoneService    str
MultipleLines   str
InternetService str
OnlineSecurity  str
OnlineBackup    str
DeviceProtection str
TechSupport     str
StreamingTV     str
StreamingMovies str
Contract        str
PaperlessBilling str
PaymentMethod   str
MonthlyCharges  float64
TotalCharges    str
Churn           str
dtype: object
```

3. Count null values per column — calculate the null percentage for each

```
print("\n" + "=" * 50)
print("Null Values")
print("=" * 50)
null_counts = df.isnull().sum()
null_percentage = (null_counts / len(df)) * 100
null_df = pd.DataFrame({
    "Null Count": null_counts,
    "Null Percentage": null_percentage.round(2)
})
print(null_df)
```

```
=====
Null Values
=====
```

	Null Count	Null Percentage
customerID	0	0.0
gender	0	0.0
SeniorCitizen	0	0.0
Partner	0	0.0
Dependents	0	0.0
tenure	0	0.0
PhoneService	0	0.0
MultipleLines	0	0.0
InternetService	0	0.0
OnlineSecurity	0	0.0
OnlineBackup	0	0.0
DeviceProtection	0	0.0
TechSupport	0	0.0
StreamingTV	0	0.0
StreamingMovies	0	0.0
Contract	0	0.0
PaperlessBilling	0	0.0
PaymentMethod	0	0.0
MonthlyCharges	0	0.0
TotalCharges	0	0.0
Churn	0	0.0

4. For each key categorical column (gender, Partner, Dependents, PhoneService, InternetService, Contract, PaymentMethod, Churn) print unique values and value counts

```
categorical_columns = [
    "gender",
    "Partner",
    "Dependents",
    "PhoneService",
    "InternetService",
    "Contract",
    "PaymentMethod",
    "Churn"
]
print("\n" + "=" * 50)
print("Categorical Columns")
```

```

print("=" * 50)
for col in categorical_columns:
    print(f"\nColumn: {col}")
    print("-" * 40)
    print("Unique Values:")
    print(df[col].unique())

    print("\nValue Counts:")
    print(df[col].value_counts())

    print("\nValue Percentages:")
    print(df[col].value_counts(normalize=True) * 100)

```

```

=====
Categorical Columns
=====

Column: gender
-----
Unique Values:
<StringArray>
['Female', 'Male']
Length: 2, dtype: str

Value Counts:
gender
Male      3555
Female    3488
Name: count, dtype: int64

Value Percentages:
gender
Male      50.47565
Female    49.52435
Name: proportion, dtype: float64

Column: Partner
-----
Unique Values:
<StringArray>
['Yes', 'No']
Length: 2, dtype: str

Value Counts:
Partner
No      3641
Yes     3402

```

- Count churned vs non-churned customers — calculate the churn rate as a percentage

```

print("\n" + "=" * 50)
print("Churn Analysis")
print("=" * 50)

churn_counts = df["Churn"].value_counts()
print(churn_counts)
churn_rate = (df["Churn"] == "Yes").mean() * 100
print(f"\nChurn Rate: {churn_rate:.2f}%")

```

```
=====
Churn Analysis
=====
```

```
Churn
```

```
No      5174
```

```
Yes      1869
```

```
Name: count, dtype: int64
```

```
Churn Rate: 26.54%
```

6. Inspect the TotalCharges column specifically: print its dtype, then try `pd.to_numeric()` with `errors="coerce"` and count how many NaN values appear

```
print("\n" + "=" * 50)
print("TotalCharges Inspection")
print("=" * 50)

print("Original dtype:")
print(df["TotalCharges"].dtype)

total_numeric = pd.to_numeric(df["TotalCharges"], errors="coerce")

print("\nNaN values after conversion:")
print(total_numeric.isna().sum())
```

```
=====
TotalCharges Inspection
=====
```

```
Original dtype:
```

```
str
```

```
NaN values after conversion:
```

```
11
```

7. Print min/max/mean for tenure, MonthlyCharges, TotalCharges

```
print("\n" + "=" * 50)
print("Numerical Summary")
print("=" * 50)

print("Tenure")
print(f"Min : {df['tenure'].min()}")
print(f"Max : {df['tenure'].max()}")
print(f"Mean: {df['tenure'].mean():.2f}")

print("\nMonthlyCharges")
print(f"Min : {df['MonthlyCharges'].min()}")
print(f"Max : {df['MonthlyCharges'].max()}")
```

```
print(f"Mean: {df['MonthlyCharges'].mean():.2f}")
```

```
print("\nTotalCharges")
```

```
print(f"Min : {total_numeric.min():.2f}")
```

```
print(f"Max : {total_numeric.max():.2f}")
```

```
print(f"Mean: {total_numeric.mean():.2f}")
```

```
=====
Numerical Summary
=====
```

```
Tenure
```

```
Min : 0
```

```
Max : 72
```

```
Mean: 32.37
```

```
MonthlyCharges
```

```
Min : 18.25
```

```
Max : 118.75
```

```
Mean: 64.76
```

```
TotalCharges
```

```
Min : 18.80
```

```
Max : 8684.80
```

```
Mean: 2283.30
```

8. Write a short profiling summary: 5 bullet points of things the data team needs to know before using this data
 - The dataset has 7,043 customer records and 21 columns, with no missing values. However, converting TotalCharges to numeric reveals 11 invalid/blank entries, which should be handled before analysis.
 - The data includes 18 categorical columns, 2 integer columns (SeniorCitizen, tenure), and 1 float column (MonthlyCharges).
 - The target variable Churn is imbalanced, with 73.46% non-churned customers and 26.54% churned customers, which should be considered when training ML models.
 - Customer tenure ranges from 0 to 72 months (average 32.37 months). MonthlyCharges range from 18.25 to 118.75, while TotalCharges range from 18.80 to 8684.80 after numeric conversion.
 - Most customers have Phone Service (90.3%), Fiber Optic Internet (44%), and Month-to-Month contracts (55%).

LAB CP2 — Build a Reusable Cleaning Utility

This section focused on data cleaning and preprocessing by creating a `CustomerCleaner` class to standardize column names, clean and convert the `TotalCharges` column, normalize binary features, handle missing values, and automate the cleaning workflow. Logging and exception handling were incorporated to improve traceability and reliability, and the cleaning process was validated by comparing the dataset before and after preprocessing.

9. Create `customer_cleaner.py` with a `CustomerCleaner` class that accepts a `DataFrame` in `__init__`

```
import logging
import pandas as pd
import re
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
class CustomerCleaner:
    def __init__(self, df):
        # Work on a copy so the original DataFrame is unchanged
        self.df = df.copy()
```

10. Implement `standardize_column_names()` — convert all column names to lowercase snake_case (`MonthlyCharges` → `monthly_charges`)

```
def standardize_column_names(self):
    new_columns = []
    for col in self.df.columns:
        col = re.sub(r'(?<!^)(?=[A-Z])', '_', col)
        col = col.lower()
        new_columns.append(col)

    self.df.columns = new_columns
    logging.info("Column names standardized to snake_case.")
```

11. Implement `fix_total_charges()` — replace blank strings with `NaN`, then convert to float using `pd.to_numeric(errors="coerce")`; log how many rows were affected

```
def fix_total_charges(self):
    try:
        blanks = (
```

```

        self.df["total_charges"].astype(str).str.strip().eq("").sum()
    )
    self.df["total_charges"] = (
        self.df["total_charges"]
        .replace(r'^\s*$', pd.NA, regex=True)
    )
    self.df["total_charges"] = pd.to_numeric(
        self.df["total_charges"],
        errors="coerce"
    )
    logging.info(
        f'Converted total_charges to float. '
        f'{blanks} blank values converted to NaN.'
    )
except Exception as e:
    logging.exception(
        f'Unexpected error while cleaning total_charges: {e}'
    )

```

12. Implement `normalize_binary_columns()` — map Yes→1 and No→0 for: partner, dependents, phone_service, paperless_billing, churn

```

def normalize_binary_columns(self):
    binary_columns = [
        "partner",
        "dependents",
        "phone_service",
        "paperless_billing",
        "churn"
    ]
    mapping = {
        "Yes": 1,
        "No": 0
    }
    for col in binary_columns:
        if col in self.df.columns:
            self.df[col] = self.df[col].map(mapping)

    logging.info("Binary columns normalized.")

```

13. Implement `handle_nulls()` — for total_charges nulls, fill with monthly_charges value (justified: new customers with tenure=0 have no total charges yet)

```

def handle_nulls(self):
    missing = self.df["total_charges"].isna().sum()

```

```

self.df["total_charges"] = self.df["total_charges"].fillna(
    self.df["monthly_charges"]
)
logging.info(
    f"Filled {missing} missing total_charges values "
    "using monthly_charges."
)

```

14. Implement `clean()` — calls all methods in sequence and returns the cleaned DataFrame

```

def clean(self):
    logging.info("Starting customer data cleaning.")
    self.standardize_column_names()
    self.fix_total_charges()
    self.normalize_binary_columns()
    self.handle_nulls()
    logging.info("Customer data cleaning completed.")
    return self.df

```

15. Add Python logging statements at each step (use `logging.INFO`, not `print()`)

```

logging.basicConfig(
    level=logging.INFO,
    format="%asctime)s - %(levelname)s - %(message)s"
)

```

16. Add try/except around `fix_total_charges()` and log any unexpected errors

```

def fix_total_charges(self):
    try:
        blanks = (
            self.df["total_charges"]
            .astype(str)
            .str.strip()
            .eq("")
            .sum()
        )
        self.df["total_charges"] = (
            self.df["total_charges"]
            .replace(r'^\s*$', pd.NA, regex=True)
        )
        self.df["total_charges"] = pd.to_numeric(
            self.df["total_charges"],
            errors="coerce"
        )
        logging.info(

```

```

        f"Converted total_charges to float. "
        f"{blanks} blank values converted to NaN."
    )
except Exception as e:
    logging.exception(
        f"Unexpected error while cleaning total_charges: {e}"
    )

```

17. Test: instantiate CustomerCleaner, call .clean(), print shape and dtypes before/after

```

import pandas as pd
from cp2_customer_cleaner import CustomerCleaner
# Load dataset
df = pd.read_csv("customer_churn.csv")

print("Before Cleaning")
print("-" * 50)
print("Shape:", df.shape)
print(df.dtypes)
# Clean
cleaner = CustomerCleaner(df)
clean_df = cleaner.clean()
print("\nAfter Cleaning")
print("-" * 50)
print("Shape:", clean_df.shape)
print(clean_df.dtypes)

print("\nFirst Five Rows")
print(clean_df.head())
...
dtype: object
2026-07-31 10:16:36,954 - INFO - Starting customer data cleaning.
2026-07-31 10:16:36,955 - INFO - Column names standardized to snake_case.
2026-07-31 10:16:36,963 - INFO - Converted total_charges to float. 11 blank values converted to NaN.
2026-07-31 10:16:36,966 - INFO - Binary columns normalized.
2026-07-31 10:16:36,966 - INFO - Filled 11 missing total_charges values using monthly_charges.
2026-07-31 10:16:36,966 - INFO - Customer data cleaning completed.

```

LAB CP3 — Generate Business Insights

This section focused on exploratory analysis of customer churn patterns using the cleaned dataset. Churn rates were calculated across contract types, internet services, payment methods, and tenure groups to identify high-risk customer segments. Additional analysis compared monthly charges for churned and non-churned customers and examined combined feature groups to provide actionable insights for customer retention strategies

18. Load and clean the data using CustomerCleaner

```
df = pd.read_csv("customer_churn.csv")
cleaner = CustomerCleaner(df)
df = cleaner.clean()
```

19. Compute churn rate by contract type — sort descending. Which contract has the highest churn?

```
print("\n===== Churn Rate by Contract =====")
contract_churn = (
    df.groupby("contract")["churn"]
    .mean()
    .sort_values(ascending=False)
    * 100
)
print(contract_churn)
highest_contract = contract_churn.idxmax()
print(f"\nHighest Churn Contract: {highest_contract}")
```

```
===== Churn Rate by Contract =====
contract
Month-to-month    42.709677
One year          11.269518
Two year           2.831858
Name: churn, dtype: float64

Highest Churn Contract: Month-to-month
```

20. Compute churn rate by internet service type

```
print("\n===== Churn Rate by Internet Service =====")
internet_churn = (
    df.groupby("internet_service")["churn"]
    .mean()
    * 100
)
print(internet_churn)
```

```

===== Churn Rate by Internet Service =====
internet_service
DSL          18.959108
Fiber optic  41.892765
No           7.404980
Name: churn, dtype: float64

```

21. Compute churn rate by payment method

```

print("\n===== Churn Rate by Payment Method =====")
payment_churn = (
    df.groupby("payment_method")["churn"]
    .mean()
    * 100
)
print(payment_churn)

```

```

===== Churn Rate by Payment Method =====
payment_method
Bank transfer (automatic)    16.709845
Credit card (automatic)    15.243101
Electronic check            45.285412
Mailed check                 19.106700
Name: churn, dtype: float64

```

22. Create a tenure_bucket column using pd.cut() with bins [0,12,24,48,72] and labels ["0-12", "13-24", "25-48", "49-72"]. Compute churn rate per bucket.

```

df["tenure_bucket"] = pd.cut(
    df["tenure"],
    bins=[0, 12, 24, 48, 72],
    labels=["0-12", "13-24", "25-48", "49-72"],
    include_lowest=True
)
print("\n===== Churn Rate by Tenure Bucket =====")
tenure_churn = (
    df.groupby("tenure_bucket")["churn"]
    .mean()
    * 100
)
print(tenure_churn)

```

```

===== Churn Rate by Tenure Bucket =====
tenure_bucket
0-12      47.438243
13-24     28.710938
25-48     20.388959
49-72      9.513176
Name: churn, dtype: float64

```

23. Compute average monthly_charges for churned vs non-churned customers

```

print("\n===== Average Monthly Charges =====")
avg_monthly = (
    df.groupby("churn")["monthly_charges"]
    .mean()
)
print(avg_monthly)

```

```

===== Average Monthly Charges =====
churn
0      61.265124
1      74.441332
Name: monthly_charges, dtype: float64

```

24. Find the combination (contract + internet_service) with the highest churn rate using groupby on two columns

```

print("\n===== Contract + Internet Service =====")
combo = (
    df.groupby(["contract", "internet_service"])["churn"]
    .mean()
    .sort_values(ascending=False)
    * 100
)
print(combo)
print("\nHighest Risk Combination:")
print(combo.head(1))

```

```

===== Contract + Internet Service =====
contract      internet_service
Month-to-month  Fiber optic      54.605263
                DSL              32.215863
One year       Fiber optic      19.294991
Month-to-month  No              18.893130
One year       DSL              9.298246
Two year       Fiber optic      7.226107
One year       No               2.472527
Two year       DSL              1.910828
                No               0.783699
Name: churn, dtype: float64

Highest Risk Combination:
contract      internet_service
Month-to-month  Fiber optic      54.605263
Name: churn, dtype: float64

```

25. Write a 3-sentence business summary: which segments are highest risk and what that means for the retention team

The analysis shows that customers on month-to-month contracts using fiber optic internet service have the highest churn rate (54.61%), making them the most at-risk customer segment. Churn is also high among customers with 0–12 months of tenure. The retention team should prioritize these high-risk segments by offering contract upgrade incentives and targeted engagement campaigns during the first year of service to reduce churn.

LAB CP4 — Build Engineered Features

This section focused on feature engineering by creating new variables from the cleaned dataset to better capture customer behavior and churn patterns. Features such as tenure groups, high monthly charges, service usage count, long-term customer status, streaming bundles, and automatic payment indicators were derived and analyzed for their relationship with churn. The enhanced dataset was then saved for use in subsequent modeling and predictive analysis.

26. Start from the cleaned DataFrame (CustomerCleaner output)

```
import pandas as pd
from cp2_customer_cleaner import CustomerCleaner
df = pd.read_csv("customer_churn.csv")
cleaner = CustomerCleaner(df)
df = cleaner.clean()
print(df.columns)
```

27. Feature 1 — tenure_bucket: pd.cut() with bins [0,12,24,48,72] as in CP3

```
df["tenure_bucket"] = pd.cut(
    df["tenure"],
    bins=[0, 12, 24, 48, 72],
    labels=["0-12", "13-24", "25-48", "49-72"],
    include_lowest=True
)
```

28. Feature 2 — high_charge_flag: 1 if monthly_charges > median monthly_charges, else 0

```
median_charge = df["monthly_charges"].median()
df["high_charge_flag"] = (
    df["monthly_charges"] > median_charge
).astype(int)
```

29. Feature 3 — service_count: count how many of these 6 service columns equal 1:
online_security, online_backup, device_protection, tech_support, streaming_tv,
streaming_movies

```
service_columns = [  
    "online_security",  
    "online_backup",  
    "device_protection",  
    "tech_support",  
    "streaming_tv",  
    "streaming_movies"  
]  
  
for col in service_columns:  
    df[col] = df[col].map({"Yes": 1, "No": 0}).fillna(0).astype(int)  
df["service_count"] = df[service_columns].sum(axis=1)
```

30. Feature 4 — is_long_term_customer: 1 if tenure >= 24, else 0

```
df["is_long_term_customer"] = (  
    df["tenure"] >= 24  
)<div data-bbox="142 519 710 557" data-label="List-Group">

31. Feature 5 — has_streaming_bundle: 1 if both streaming_tv=1 AND  
streaming_movies=1, else 0



```
df["has_streaming_bundle"] = (
 (df["streaming_tv"] == 1) &
 (df["streaming_movies"] == 1)
)<div data-bbox="142 651 834 690" data-label="List-Group">

32. Feature 6 — auto_pay_flag: 1 if payment_method contains "automatic", else 0 (use
str.contains())


```
df["auto_pay_flag"] = (  
    df["payment_method"]  
    .str.contains("automatic", case=False, na=False)  
)<div data-bbox="142 784 675 801" data-label="List-Group">

33. Print the correlation of each new feature with the churn column



```
print("\nCorrelation with Churn\n")

features = [
 "high_charge_flag",
 "service_count",
```


```


```


```

```

"is_long_term_customer",
"has_streaming_bundle",
"auto_pay_flag"
]
for feature in features:
    corr = df[feature].corr(df["churn"])
    print(f'{feature:25s}: {corr:.4f}')

```

```

Correlation with Churn

high_charge_flag      : 0.1957
service_count         : -0.0877
is_long_term_customer : -0.3115
has_streaming_bundle  : 0.0404
auto_pay_flag         : -0.2099

```

34. Save to customer_features.csv — include all original cleaned columns plus the 6 new ones

```

df.to_csv("customer_features.csv", index=False)
print("customer_features.csv saved successfully.")
customer_features.csv saved successfully.

```

LAB CP5 — Build End-to-End Preprocessing Pipeline

This section focused on building an automated data processing pipeline that integrated data loading, cleaning, feature engineering, validation, and output generation

35. Create customer_pipeline.py
36. Import CustomerCleaner from customer_cleaner.py and build_features() logic
37. Define load_data(filepath) — reads CSV and logs row count
38. Define build_features(df) — runs all 6 feature engineering steps from CP4 and returns the enriched DataFrame
39. Define save_outputs(clean_df, feature_df, output_dir) — saves both files with timestamps in filenames
40. Define main() — calls load → clean → build_features → save in sequence
41. Add logging throughout: start time, step completion, row counts, output file paths
42. Wrap each step in try/except — if any step fails, log the error and raise; do not silently continue
43. Add a simple data quality check before saving: assert no nulls in monthly_charges, assert churn column has only 0/1 values
44. Run the script end-to-end: python customer_pipeline.py

```

import os
import logging

```

```

from datetime import datetime
import pandas as pd
from cp2_customer_cleaner import CustomerCleaner
from cp4_feat_engg import build_features

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

def load_data(filepath):
    try:
        df = pd.read_csv(filepath)
        logging.info(f"Successfully loaded {len(df)} rows from '{filepath}'")
        return df
    except Exception as e:
        logging.exception("Error while loading data.")
        Raise

def save_outputs(clean_df, feature_df, output_dir):
    try:
        os.makedirs(output_dir, exist_ok=True)
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        clean_file = os.path.join(
            output_dir,
            f"customer_clean_{timestamp}.csv"
        )
        feature_file = os.path.join(
            output_dir,
            f"customer_features_{timestamp}.csv"
        )
        clean_df.to_csv(clean_file, index=False)
        feature_df.to_csv(feature_file, index=False)
        logging.info(f"Clean dataset saved to: {clean_file}")
        logging.info(f"Feature dataset saved to: {feature_file}")
    except Exception:
        logging.exception("Error while saving output files.")
        raise

def main():
    logging.info("=" * 50)
    logging.info("Customer ETL Pipeline Started")
    logging.info("=" * 50)
    try:

```

```

raw_df = load_data("customer_churn.csv")
try:
    cleaner = CustomerCleaner(raw_df)
    clean_df = cleaner.clean()
    logging.info(
        f"Cleaning completed successfully. Rows: {len(clean_df)}"
    )
except Exception:
    logging.exception("Cleaning step failed.")
    raise

try:
    feature_df = build_features(clean_df.copy())

    logging.info(
        f"Feature engineering completed. Rows: {len(feature_df)}"
    )
except Exception:
    logging.exception("Feature engineering failed.")
    raise

try:
    assert (
        feature_df["monthly_charges"].isnull().sum() == 0
    ), "monthly_charges contains NULL values."
    assert (
        set(feature_df["churn"].unique()).issubset({0, 1})
    ), "churn column contains values other than 0 and 1."

    logging.info("Data quality checks passed.")

except Exception:
    logging.exception("Data quality validation failed.")
    raise

try:
    save_outputs(
        clean_df,
        feature_df,
        "output"
    )
except Exception:
    logging.exception("Saving outputs failed.")
    raise

logging.info("=" * 50)
logging.info("Pipeline Completed Successfully")
logging.info("=" * 50)

```

```

except Exception:
    logging.exception("Pipeline execution failed.")
    raise
if __name__ == "__main__":
    main()

```

```

2026-07-31 12:03:56,858 - INFO - =====
2026-07-31 12:03:56,859 - INFO - Customer ETL Pipeline Started
2026-07-31 12:03:56,859 - INFO - =====
2026-07-31 12:03:56,885 - INFO - Successfully loaded 7043 rows from 'customer_churn.csv'
2026-07-31 12:03:56,887 - INFO - Starting customer data cleaning.
2026-07-31 12:03:56,887 - INFO - Column names standardized to snake_case.
2026-07-31 12:03:56,897 - INFO - Converted total_charges to float. 11 blank values converted to NaN.
2026-07-31 12:03:56,900 - INFO - Binary columns normalized.
2026-07-31 12:03:56,901 - INFO - Filled 11 missing total_charges values using monthly_charges.
2026-07-31 12:03:56,901 - INFO - Customer data cleaning completed.
2026-07-31 12:03:56,901 - INFO - Cleaning completed successfully. Rows: 7043

Correlation with Churn

high_charge_flag      : 0.1957
service_count         : -0.0877
is_long_term_customer : -0.3115
has_streaming_bundle  : 0.0404
auto_pay_flag         : -0.2099
2026-07-31 12:03:56,929 - INFO - Feature engineering completed. Rows: 7043
2026-07-31 12:03:56,931 - INFO - Data quality checks passed.
2026-07-31 12:03:57,024 - INFO - Clean dataset saved to: output\customer_clean_20260731_120356.csv
2026-07-31 12:03:57,025 - INFO - Feature dataset saved to: output\customer_features_20260731_120356.csv
2026-07-31 12:03:57,025 - INFO - =====
2026-07-31 12:03:57,025 - INFO - Pipeline Completed Successfully
2026-07-31 12:03:57,025 - INFO - =====

```

PHASE 2 - DATABASE LAYER

LAB SQL1 — Design the Relational Schema

This section focused on database modeling and schema design for the customer churn dataset.

45. List all 21 columns from the dataset and categorise each as: stable identity, service subscription, billing/charges, or churn outcome

Column	Category
customerID	Stable Identity
gender	Stable Identity
SeniorCitizen	Stable Identity
Partner	Stable Identity
Dependents	Stable Identity
tenure	Service Subscription
PhoneService	Service Subscription
MultipleLines	Service Subscription
InternetService	Service Subscription
OnlineSecurity	Service Subscription
OnlineBackup	Service Subscription
DeviceProtection	Service Subscription
TechSupport	Service Subscription
StreamingTV	Service Subscription
StreamingMovies	Service Subscription
Contract	Service Subscription

PaperlessBilling	Billing / Charges
PaymentMethod	Billing / Charges
MonthlyCharges	Billing / Charges
TotalCharges	Billing / Charges
Churn	Churn Outcome

46. Define the entities: customers (identity), contracts (subscription terms), services (what they subscribe to), billing (charges), customer_status (churn)

Customers (Identity): customer_id, gender, senior_citizen, partner, dependents

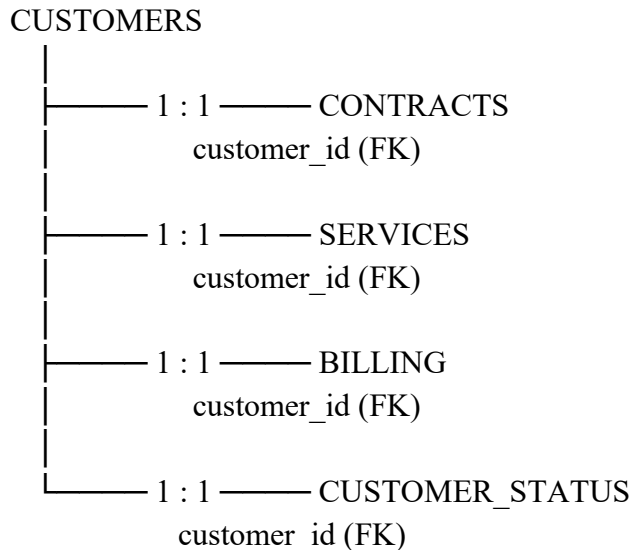
Contracts (Subscription Terms): customer_id, contract, tenure, phone_service, multiple_lines, internet_service

Services: customer_id, online_security, online_backup, device_protection, tech_support, streaming_tv, streaming_movies

Billing: customer_id, paperless_billing, payment_method, monthly_charges, total_charges

Customer Status: customer_id, churn

47. Draw the entity-relationship diagram: which tables have foreign keys to which?



48. Define the primary key for each table (customer_id is the natural key throughout)

The primary key for each table is customer_id

49. Identify which columns are dimension-like (contract type, payment method, internet service) vs measure-like (tenure, monthly_charges, total_charges)

Dimension-like (categorical): Contract, PaymentMethod, InternetService, PhoneService, MultipleLines, Gender, Partner, Dependents, PaperlessBilling, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies

Measure-like(numerical): tenure, MonthlyCharges, TotalCharges, SeniorCitizen

50. Write a CREATE TABLE DDL statement for each of the 4–5 tables

```
CREATE TABLE customers (  
    customer_id VARCHAR(20) PRIMARY KEY,  
    gender VARCHAR(10),  
    senior_citizen INT,  
    partner INT,  
    dependents INT
```

```
);
```

```
CREATE TABLE contracts (  
    customer_id VARCHAR(20),  
    contract VARCHAR(30),  
    tenure INT,  
    phone_service INT,  
    multiple_lines VARCHAR(30),  
    internet_service VARCHAR(30),  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)
```

```
);
```

```
CREATE TABLE services (  
    customer_id VARCHAR(20),  
    online_security VARCHAR(30),  
    online_backup VARCHAR(30),  
    device_protection VARCHAR(30),  
    tech_support VARCHAR(30),  
    streaming_tv VARCHAR(30),  
    streaming_movies VARCHAR(30),  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)
```

```
);
```

```
CREATE TABLE billing (  
    customer_id VARCHAR(20),  
    paperless_billing INT,  
    payment_method VARCHAR(50),  
    monthly_charges DECIMAL(10,2) ,  
    total_charges DECIMAL(10,2),  
    FOREIGN KEY (customer_id)  
    REFERENCES customers(customer_id)
```

```
);
```

```
CREATE TABLE customer_status (
  customer_id VARCHAR(20),
  churn INT,
  FOREIGN KEY (customer_id)
  REFERENCES customers(customer_id)
);
```

51. Discuss with trainer: should this be a normalised OLTP schema or a denormalised analytics schema? Why?

For this telecom customer management system, a normalized OLTP schema is the better choice because customer, subscription, and billing information are maintained separately and updated independently

LAB SQL2 — Load Raw Data into Staging

The raw data is loaded into the staging table in MySQL

52. Create a staging table stg_customer_raw with columns matching the raw CSV exactly (TotalCharges as VARCHAR, not FLOAT)

```
CREATE TABLE stg_customer_raw (

  customerID VARCHAR(20),
  gender VARCHAR(10),
  SeniorCitizen INT,
  Partner VARCHAR(5),
  Dependents VARCHAR(5),
  tenure INT,
  PhoneService VARCHAR(5),
  MultipleLines VARCHAR(30),
  InternetService VARCHAR(30),
  OnlineSecurity VARCHAR(30),
  OnlineBackup VARCHAR(30),
  DeviceProtection VARCHAR(30),
  TechSupport VARCHAR(30),
  StreamingTV VARCHAR(30),
  StreamingMovies VARCHAR(30),
  Contract VARCHAR(30),
  PaperlessBilling VARCHAR(5),
  PaymentMethod VARCHAR(50),
```

```
MonthlyCharges DECIMAL(10,2),
TotalCharges VARCHAR(30),
Churn VARCHAR(5)
```

```
);
```

53. Load the CSV into the staging table using either: Python + SQLAlchemy to `_sql()`, or a MySQL LOAD DATA INFILE command
import pandas as pd

```
import pandas as pd
from sqlalchemy import create_engine
engine = create_engine(
    "mysql+pymysql://root:root@localhost/telecom_db"
)
df = pd.read_csv("customer_churn.csv")
df.to_sql(
    "stg_customer_raw",
    engine,
    if_exists="append",
    index=False
)
print("Data loaded successfully.")
```

54. Verify row count: `SELECT COUNT(*) FROM stg_customer_raw` — should be 7043
`SELECT COUNT(*) FROM stg_customer_raw;`

```
mysql> SELECT COUNT(*) FROM stg_customer_raw;
+-----+
| COUNT(*) |
+-----+
|      7043 |
+-----+
1 row in set (0.01 sec)
```

55. Inspect TotalCharges in the staging table: `SELECT TotalCharges FROM stg_customer_raw WHERE TotalCharges = "" LIMIT 5`
`SELECT TotalCharges FROM stg_customer_raw WHERE TotalCharges = "" LIMIT 5;`

```
mysql> SELECT TotalCharges
-> FROM stg_customer_raw
-> WHERE TotalCharges = ""
-> LIMIT 5;
Empty set (0.02 sec)
```

56. Inspect data types: confirm TotalCharges is stored as text in staging (not yet converted)
`DESCRIBE stg_customer_raw;`

PaymentMethod	varchar(30)	YES	
MonthlyCharges	decimal(10,2)	YES	
TotalCharges	varchar(30)	YES	
Churn	varchar(5)	YES	

57. Record the load time and row count in a simple ingestion_log table

```
CREATE TABLE ingestion_log (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  table_name VARCHAR(50),
  load_time DATETIME,
  row_count INT
);
INSERT INTO ingestion_log
(table_name, load_time, row_count)
VALUES
(
'stg_customer_raw',
NOW(),
(SELECT COUNT(*) FROM stg_customer_raw)
);
SELECT * FROM ingestion_log;
```

log_id	table_name	load_time	row_count
1	stg_customer_raw	2026-08-07 10:18:36	7043

1 row in set (0.00 sec)

58. Run a basic validation: count distinct customerIDs — should equal row count (no duplicates)

```
SELECT COUNT(DISTINCT customerID) FROM stg_customer_raw;
```

COUNT(DISTINCT customerID)
7043

1 row in set (0.07 sec)

LAB SQL3 — Build Curated Tables + SQLAlchemy ORM

Built the curated tables — clean, typed, and structured — that the rest of the system will actually use

59. Create dim_contract with rows for each unique contract type (Month-to-month, One year, Two year)

```
CREATE TABLE dim_contract (
  contract_id INT AUTO_INCREMENT PRIMARY KEY,
```

```

        contract_name VARCHAR(30) UNIQUE
    );
    INSERT INTO dim_contract (contract_name)
    SELECT DISTINCT Contract
    FROM stg_customer_raw;
    SELECT * FROM dim_contract;

```

```

mysql> SELECT * FROM dim_contract;
+-----+-----+
| contract_id | contract_name |
+-----+-----+
|          1 | Month-to-month |
|          2 | One year       |
|          3 | Two year       |
+-----+-----+

```

60. Create dim_payment with rows for each payment method

```

CREATE TABLE dim_payment (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    payment_method VARCHAR(50) UNIQUE
);
INSERT INTO dim_payment (payment_method)
SELECT DISTINCT PaymentMethod
FROM stg_customer_raw;
SELECT * FROM dim_payment;

```

```

mysql> SELECT * FROM dim_payment;
+-----+-----+
| payment_id | payment_method |
+-----+-----+
|          3 | Bank transfer (automatic) |
|          4 | Credit card (automatic) |
|          1 | Electronic check |
|          2 | Mailed check |
+-----+-----+
4 rows in set (0.00 sec)

```

61. Create the customers table: INSERT from staging, standardising gender, casting senior_citizen, binary-encoding partner/dependents

```

CREATE TABLE customers (
    customer_id VARCHAR(20) PRIMARY KEY,
    gender VARCHAR(10),
    senior_citizen INT,
    partner TINYINT,

```

```

        dependents TINYINT
    );
INSERT INTO customers
(
    customer_id,
    gender,
    senior_citizen,
    partner,
    dependents
)

SELECT
customerID,
gender,
CAST(SeniorCitizen AS UNSIGNED),

CASE
WHEN Partner='Yes' THEN 1
ELSE 0
END,

CASE
WHEN Dependents='Yes' THEN 1
ELSE 0
END

FROM stg_customer_raw;

```

62. Create fact_customer_account: INSERT tenure, charges, churn — cast TotalCharges with NULLIF to handle blanks, join to dim tables for foreign keys

```

CREATE TABLE fact_customer_account (
    customer_id VARCHAR(20),
    contract_id INT,
    payment_id INT,
    tenure INT,
    monthly_charges DECIMAL(10,2),
    total_charges DECIMAL(10,2),

```

```

        churn TINYINT,
        PRIMARY KEY(customer_id),
        FOREIGN KEY(customer_id)
            REFERENCES customers(customer_id),
        FOREIGN KEY(contract_id)
            REFERENCES dim_contract(contract_id),
        FOREIGN KEY(payment_id)
            REFERENCES dim_payment(payment_id)

);
INSERT INTO fact_customer_account
(
    customer_id,
    contract_id,
    payment_id,
    tenure,
    monthly_charges,
    total_charges,
    churn
)

SELECT
    s.customerID,
    c.contract_id,
    p.payment_id,
    s.tenure,
    s.MonthlyCharges,

    CAST(NULLIF(TRIM(s.TotalCharges),'') AS DECIMAL(10,2)),

    CASE
        WHEN s.Churn='Yes' THEN 1
        ELSE 0
    END

FROM stg_customer_raw s

JOIN dim_contract c
ON s.Contract = c.contract_name

```

```
JOIN dim_payment p
ON s.PaymentMethod = p.payment_method;
```

63. Verify counts: each curated table row count should match expectations

```
SELECT COUNT(*) FROM customers;
```

```
mysql> SELECT COUNT(*) FROM customers;
+-----+
| COUNT(*) |
+-----+
|      7043 |
+-----+
1 row in set (0.02 sec)
```

```
SELECT COUNT(*) FROM fact_customer_account;
```

```
mysql> SELECT COUNT(*) FROM fact_customer_account;
+-----+
| COUNT(*) |
+-----+
|      7043 |
+-----+
1 row in set (0.05 sec)
```

```
SELECT COUNT(*) FROM dim_contract;
```

```
mysql> SELECT COUNT(*) FROM dim_contract;
+-----+
| COUNT(*) |
+-----+
|         3 |
+-----+
1 row in set (0.00 sec)
```

```
SELECT COUNT(*) FROM dim_payment;
```

```
mysql> SELECT COUNT(*) FROM dim_payment;
+-----+
| COUNT(*) |
+-----+
|         4 |
+-----+
1 row in set (0.00 sec)
```

64. Run a JOIN query across customers + fact_customer_account + dim_contract to confirm the relationships work

```
SELECT
c.customer_id,
c.gender,

f.tenure,
d.contract_name,
```

```
f.monthly_charges
f.churn
```

```
FROM customers c
JOIN fact_customer_account
ON c.customer_id=f.customer_id
JOIN dim_contract d
ON f.contract_id=d.contract_id
LIMIT 10;
```

customer_id	gender	tenure	contract_name	monthly_charges	churn
0003-MKNFE	Male	9	Month-to-month	59.90	0
0004-TLHLJ	Male	4	Month-to-month	73.90	1
0011-IGKFF	Male	13	Month-to-month	98.00	1
0013-EXCHZ	Female	3	Month-to-month	83.90	1
0013-MHZWF	Female	9	Month-to-month	69.40	0
0015-UOCOJ	Female	7	Month-to-month	48.20	0
0018-NYROU	Female	5	Month-to-month	68.95	0
0021-IKXGC	Female	1	Month-to-month	72.10	0
0023-HGHWL	Male	1	Month-to-month	25.10	1
0023-XUOPT	Female	13	Month-to-month	94.10	1

65. Introduce SQLAlchemy: create Python model classes for at least two of these tables (Customer, Contract)

```
from sqlalchemy.orm import declarative_base
from sqlalchemy import Column, Integer, String

Base = declarative_base()

class Customer(Base):
    __tablename__ = "customers"
    customer_id = Column(String(20), primary_key=True)
    gender = Column(String(10))
    senior_citizen = Column(Integer)
    partner = Column(Integer)
    dependents = Column(Integer)

class Contract(Base):
    __tablename__ = "dim_contract"
    contract_id = Column(Integer, primary_key=True)
    contract_name = Column(String(30))
```

66. Use a SQLAlchemy session to query one customer by customerID and print the result

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine(
    "mysql+pymysql://root:root@localhost/telecom_db"
)

Session = sessionmaker(bind=engine)

session = Session()

customer = session.query(Customer).filter_by(
    customer_id="7590-VHVEG"
).first()

print(customer.customer_id)
print(customer.gender)
print(customer.partner)

```

```

PS C:\Users\yuvasree.r\Desktop\python test> & C:\U
"c:/Users/yuvasree.r/Desktop/python test/sql3_.py"
7590-VHVEG
Female
1

```

LAB SQL4 — Write Business Analytics Queries

Wrote sql queries to answer business questions

67. Query 1 — Churn rate by contract: SELECT contract_type, AVG(churn) AS churn_rate FROM fact_customer_account JOIN dim_contract GROUP BY contract_type ORDER BY churn_rate DESC

```

SELECT
    d.contract_name AS contract_type,
    AVG(f.churn) * 100 AS churn_rate
FROM fact_customer_account f
JOIN dim_contract d
    ON f.contract_id = d.contract_id
GROUP BY d.contract_name
ORDER BY churn_rate DESC;

```

contract_type	churn_rate
Month-to-month	42.7100
One year	11.2700
Two year	2.8300

3 rows in set (0.07 sec)

68. Query 2 — Churn rate by payment method: similar pattern, join with dim_payment

```
SELECT
    p.payment_method,
    AVG(f.churn) * 100 AS churn_rate
FROM fact_customer_account f
JOIN dim_payment p
    ON f.payment_id = p.payment_id
GROUP BY p.payment_method
ORDER BY churn_rate DESC;
```

payment_method	churn_rate
Electronic check	45.2900
Mailed check	19.1100
Bank transfer (automatic)	16.7100
Credit card (automatic)	15.2400

4 rows in set (0.01 sec)

69. Query 3 — Average monthly charges by churn status: GROUP BY churn, return avg(monthly_charges)

```
SELECT
    churn,
    AVG(monthly_charges) AS avg_monthly_charges
FROM fact_customer_account
GROUP BY churn;
```

churn	avg_monthly_charges
0	61.265124
1	74.441332

2 rows in set (0.01 sec)

70. Query 4 — Top risky segments: customers with month-to-month contract AND tenure < 12 AND monthly_charges > average — count and list

```

SELECT
    COUNT(*) AS risky_customer_count
FROM fact_customer_account f
JOIN dim_contract d
    ON f.contract_id = d.contract_id
WHERE d.contract_name = 'Month-to-month'
    AND f.tenure < 12
    AND f.monthly_charges > (
        SELECT AVG(monthly_charges)
        FROM fact_customer_account
    );

```

```

+-----+
| risky_customer_count |
+-----+
|                935 |
+-----+
1 row in set (0.04 sec)

```

```

SELECT
    f.customer_id,
    d.contract_name,
    f.tenure,
    f.monthly_charges,
    f.churn
FROM fact_customer_account f
JOIN dim_contract d
    ON f.contract_id = d.contract_id
WHERE d.contract_name = 'Month-to-month'
    AND f.tenure < 12
    AND f.monthly_charges > (
        SELECT AVG(monthly_charges)
        FROM fact_customer_account
    )
ORDER BY f.monthly_charges DESC;

```

customer_id	contract_name	tenure	monthly_charges	churn
3292-PBZEJ	Month-to-month	11	111.40	0
9851-KIELU	Month-to-month	10	110.10	1
3992-YWPKO	Month-to-month	6	109.90	1
2081-VEYEH	Month-to-month	3	107.95	0
5760-IFJOZ	Month-to-month	3	107.95	0
1400-MMYXY	Month-to-month	3	105.90	1
3389-YGYAI	Month-to-month	8	105.50	1
5052-PNLOS	Month-to-month	3	105.35	1
4587-VVTOX	Month-to-month	6	105.30	1
6734-GMPVK	Month-to-month	5	105.30	0
6496-SLWHQ	Month-to-month	3	105.00	1

71. Query 5 — High-value churned customers: churned customers with total_charges > 2000 — how many and what is their average tenure?

```
SELECT
    COUNT(*) AS high_value_churned_customers,
    AVG(tenure) AS average_tenure
FROM fact_customer_account
WHERE churn = 1
AND total_charges > 2000;
```

high_value_churned_customers	average_tenure
530	44.3170

1 row in set (0.01 sec)

72. For each query: first state the business question in plain English, then write the SQL, then interpret the result in one sentence

1 Which contract has highest churn?

GROUP BY contract_name

Month-to-month contracts have the highest churn rate, while two-year contracts have the lowest churn.

2 Which payment method has highest churn?

GROUP BY payment_method

Electronic check has the highest churn rate, indicating that customers using this payment method are a high-risk segment.

3 Do churned customers pay more?

GROUP BY churn

Customers who churned have higher average monthly charges than customers who stayed.

4 How many high-risk customers exist and who are they?

WHERE + subquery AVG()

This identifies month-to-month customers paying above-average charges, who should be prioritized by the retention team.

5 How many high-value customers churned?

churn = 1 AND total_charges > 2000

This identifies valuable customers who have already churned, helping the retention team understand the potential revenue loss and target similar customers before they leave.

73. Bonus: write a single query that returns contract type, avg monthly charges, avg tenure, and churn rate together — sorted by churn rate

```
SELECT
    d.contract_name AS contract_type,
    AVG(f.monthly_charges) AS avg_monthly_charges,
    AVG(f.tenure) AS avg_tenure,
    AVG(f.churn) * 100 AS churn_rate
FROM fact_customer_account f
JOIN dim_contract d
    ON f.contract_id = d.contract_id
GROUP BY d.contract_name
ORDER BY churn_rate DESC;
```

contract_type	avg_monthly_charges	avg_tenure	churn_rate
Month-to-month	66.398490	18.0366	42.7100
One year	65.048608	42.0448	11.2700
Two year	60.770413	56.7351	2.8300

LAB SQL5 — Create ML Feature Table

74. Design the customer_ml_features table schema: all features from CP4 + original numeric columns + churn label
75. Create the table in SQL with appropriate types (INT for flags/counts, FLOAT for charges, VARCHAR for buckets)
76. Populate it using INSERT ... SELECT from fact_customer_account + dim tables, computing engineered features in SQL:
77. — service_count: SUM of 6 service binary columns
78. — high_charge_flag: CASE WHEN monthly_charges > (SELECT AVG...) THEN 1 ELSE 0 END
79. — is_long_term_customer: CASE WHEN tenure >= 24 THEN 1 ELSE 0 END
80. — auto_pay_flag: CASE WHEN payment_method LIKE "%automatic%" THEN 1 ELSE 0 END

```

CREATE TABLE customer_ml_features (
    customer_id VARCHAR(20) PRIMARY KEY,
    tenure INT,
    monthly_charges FLOAT,
    total_charges FLOAT,

    tenure_bucket VARCHAR(10),
    service_count INT,
    high_charge_flag INT,
    is_long_term_customer INT,
    has_streaming_bundle INT,
    auto_pay_flag INT,

    churn INT
);
INSERT INTO customer_ml_features (
    customer_id,
    tenure,
    monthly_charges,
    total_charges,
    tenure_bucket,
    service_count,
    high_charge_flag,
    is_long_term_customer,
    has_streaming_bundle,
    auto_pay_flag,
    churn
)
SELECT
    f.customer_id,
    f.tenure,
    CAST(f.monthly_charges AS FLOAT),
    CAST(f.total_charges AS FLOAT),

    CASE
        WHEN f.tenure BETWEEN 0 AND 12 THEN '0-12'
        WHEN f.tenure BETWEEN 13 AND 24 THEN '13-24'
        WHEN f.tenure BETWEEN 25 AND 48 THEN '25-48'

```

```
    WHEN f.tenure BETWEEN 49 AND 72 THEN '49-72'  
END AS tenure_bucket,
```

```
(  
    CASE WHEN s.PhoneService = 'Yes' THEN 1 ELSE 0 END +  
    CASE WHEN s.OnlineSecurity = 'Yes' THEN 1 ELSE 0 END +  
    CASE WHEN s.OnlineBackup = 'Yes' THEN 1 ELSE 0 END +  
    CASE WHEN s.DeviceProtection = 'Yes' THEN 1 ELSE 0 END +  
    CASE WHEN s.TechSupport = 'Yes' THEN 1 ELSE 0 END +  
    CASE WHEN s.StreamingTV = 'Yes' THEN 1 ELSE 0 END  
) AS service_count,
```

```
CASE  
    WHEN f.monthly_charges > (  
        SELECT AVG(monthly_charges)  
        FROM fact_customer_account  
    )  
    THEN 1  
    ELSE 0  
END AS high_charge_flag,
```

```
CASE  
    WHEN f.tenure >= 24 THEN 1  
    ELSE 0  
END AS is_long_term_customer,
```

```
CASE  
    WHEN s.StreamingTV = 'Yes'  
    AND s.StreamingMovies = 'Yes'  
    THEN 1  
    ELSE 0  
END AS has_streaming_bundle,
```

```
CASE  
    WHEN dp.payment_method LIKE '%automatic%' THEN 1  
    ELSE 0  
END AS auto_pay_flag,
```

```
f.churn
```

```

FROM fact_customer_account f
JOIN dim_payment dp
  ON f.payment_id = dp.payment_id
JOIN stg_customer_raw s
  ON f.customer_id = s.customerID;

```

81. Verify: SELECT COUNT(*) should be 7043; SELECT COUNT(*) WHERE monthly_charges IS NULL should be 0
 SELECT COUNT(*) AS null_monthly_charges
 FROM customer_ml_features
 WHERE monthly_charges IS NULL;

```

+-----+
| null_monthly_charges |
+-----+
|                      0 |
+-----+
1 row in set (0.00 sec)

```

82. Run a final check: SELECT churn, COUNT(*) FROM customer_ml_features GROUP BY churn — confirm class distribution
 SELECT churn, COUNT(*) AS customer_count
 FROM customer_ml_features
 GROUP BY churn;

```

+-----+-----+
| churn | customer_count |
+-----+-----+
|      0 |           5174 |
|      1 |           1869 |
+-----+-----+
2 rows in set (0.00 sec)

```

LAB SQL6 — Create a Rule-Based High-Risk View

Created High-Risk table to identify customers who are most likely to churn

83. Define the business rules for "high risk": month-to-month contract AND tenure < 12 AND monthly_charges > dataset average

A customer is high risk when all three conditions are true:

Contract is Month-to-month

Tenure is less than 12 months

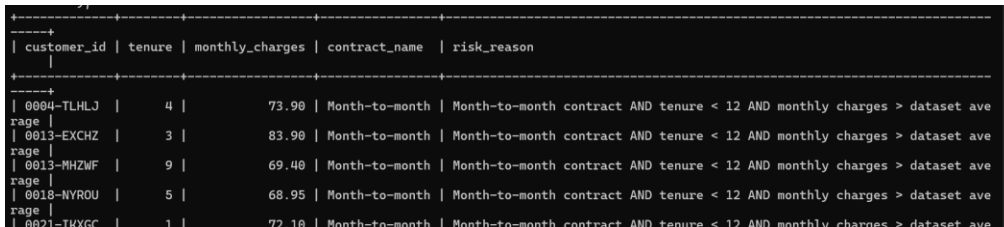
Monthly charges are greater than the dataset average

contract_name = 'Month-to-month'

```
AND tenure < 12
AND monthly_charges > dataset_average
```

84. Write a SELECT query that returns matching customers with a risk_reason string column explaining why each was flagged

```
SELECT
    f.customer_id,
    f.tenure,
    f.monthly_charges,
    dc.contract_name,
    'Month-to-month contract AND tenure < 12 AND monthly charges > dataset average'
    AS risk_reason
FROM fact_customer_account f
JOIN dim_contract dc
    ON f.contract_id = dc.contract_id
WHERE dc.contract_name = 'Month-to-month'
    AND f.tenure < 12
    AND f.monthly_charges > (
        SELECT AVG(monthly_charges)
        FROM fact_customer_account
    );
```



customer_id	tenure	monthly_charges	contract_name	risk_reason
0004-TLHLJ	4	73.90	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0013-EXCHZ	3	83.90	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0013-MHZWF	9	69.40	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0018-NYROU	5	68.95	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0021-IXXGC	1	72.10	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave

85. Test the query — verify count and inspect sample rows

```
SELECT COUNT(*) AS high_risk_count
FROM fact_customer_account f
JOIN dim_contract dc
    ON f.contract_id = dc.contract_id
WHERE dc.contract_name = 'Month-to-month'
    AND f.tenure < 12
    AND f.monthly_charges > (
        SELECT AVG(monthly_charges)
        FROM fact_customer_account
    );
```

```

+-----+
| high_risk_count |
+-----+
|          935 |
+-----+
1 row in set (0.08 sec)

```

```

SELECT
    f.customer_id,
    f.tenure,
    f.monthly_charges,
    dc.contract_name
FROM fact_customer_account f
JOIN dim_contract dc
    ON f.contract_id = dc.contract_id
WHERE dc.contract_name = 'Month-to-month'
    AND f.tenure < 12
    AND f.monthly_charges > (
        SELECT AVG(monthly_charges)
        FROM fact_customer_account
    )
LIMIT 10;

```

```

+-----+-----+-----+-----+
| customer_id | tenure | monthly_charges | contract_name |
+-----+-----+-----+-----+
| 0004-TLHLJ  | 4      | 73.90           | Month-to-month |
| 0013-EXCHZ  | 3      | 83.90           | Month-to-month |
| 0013-MHZWF  | 9      | 69.40           | Month-to-month |
| 0018-NYROU  | 5      | 68.95           | Month-to-month |
| 0021-IKXGC  | 1      | 72.10           | Month-to-month |
| 0094-OIFMO  | 11     | 95.00           | Month-to-month |
| 0107-YHINA  | 1      | 99.75           | Month-to-month |
| 0122-OAHPZ  | 7      | 73.85           | Month-to-month |
| 0133-BMFZO  | 2      | 86.25           | Month-to-month |
| 0137-OCGAB  | 1      | 80.20           | Month-to-month |
+-----+-----+-----+-----+
10 rows in set (0.05 sec)

```

86. Create the view: CREATE VIEW v_high_risk_customers AS ...
 CREATE OR REPLACE VIEW v_high_risk_customers AS
 SELECT
 f.customer_id,
 f.tenure,
 f.monthly_charges,
 dc.contract_name,
 'Month-to-month contract AND tenure < 12 AND monthly charges > dataset average'

```

        AS risk_reason
FROM fact_customer_account f
JOIN dim_contract dc
    ON f.contract_id = dc.contract_id
WHERE dc.contract_name = 'Month-to-month'
    AND f.tenure < 12
    AND f.monthly_charges > (
        SELECT AVG(monthly_charges)
        FROM fact_customer_account
    );

```

87. Query the view: SELECT * FROM v_high_risk_customers LIMIT 10
 SELECT *
 FROM v_high_risk_customers
 LIMIT 10;

customer_id	tenure	monthly_charges	contract_name	risk_reason
0004-TLHLJ	4	73.90	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0013-EXCHZ	3	83.90	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0013-RHZWF	9	69.40	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0018-NYROU	5	68.95	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0021-IXXGC	1	72.10	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0094-OIFMO	11	95.00	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0107-YHINA	1	99.75	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0122-0AHPZ	7	73.85	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0133-BMFZO	2	86.25	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave
0137-OCGAB	1	80.20	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset ave

88. Count how many customers the view flags — what percentage of the total is this?

```

SELECT COUNT(*) AS high_risk_count
FROM v_high_risk_customers;

```

high_risk_count
935
1 row in set (0.03 sec)

```

SELECT COUNT() AS high_risk_count, ROUND( COUNT() * 100.0 / (SELECT COUNT(*)
FROM fact_customer account) AS high_risk_percentage FROM v_high_risk_customers;

```

high_risk_count	high_risk_percentage
935	13.28
1 row in set (0.13 sec)	

89. Discuss: what are the limitations of rule-based risk scoring vs ML-based scoring? What does ML add?

A rule-based approach is very easy to understand, fast to implement, and allows business rules to be changed easily. However, it uses only a limited set of conditions, completely excludes customers who fail even one condition despite potentially being at risk of churn, and cannot capture complex relationships between multiple features. Machine Learning addresses these limitations by considering many features simultaneously, enabling the retention team to assess customer churn risk more accurately and prioritize customers based on their likelihood of churning rather than treating all customers who meet a fixed rule in the same way.

PHASE 3 - FASTAPI BACKEND

LAB API1 — Customer Profile Endpoint

This section focused on developing and testing a REST API using FastAPI to expose customer data from the churn analytics database. Database connectivity was established using SQLAlchemy, and a Pydantic response model was defined to ensure structured API output.

90. Create main.py with a FastAPI app instance
91. Create a SQLAlchemy engine and session factory pointing to your database
92. Define a Pydantic CustomerResponse model with the fields to return: customer_id, tenure, contract_type, internet_service, monthly_charges, churn
93. Implement GET /customer/{customer_id}: query the customers + fact_customer_account + dim_contract tables, raise HTTPException(404) if not found
94. Return the result as the CustomerResponse model
95. Run the server: uvicorn main:app --reload

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from sqlalchemy import create_engine, text
from sqlalchemy.orm import sessionmaker

app = FastAPI(title="Telecom Customer API")

DATABASE_URL = "mysql+pymysql://root:root@localhost:3306/telecom_db"

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

class CustomerResponse(BaseModel):
    customer_id: str
    tenure: int
    contract_type: str
    internet_service: str
    monthly_charges: float
    churn: int
```

```

@app.get("/customer/{customer_id}", response_model=CustomerResponse)
def get_customer(customer_id: str):

    db = SessionLocal()

    try:
        query = text("""
            SELECT
                c.customer_id,
                f.tenure,
                dc.contract_name AS contract_type,
                s.InternetService AS internet_service,
                f.monthly_charges,
                f.churn
            FROM customers c
            JOIN fact_customer_account f
              ON c.customer_id = f.customer_id
            JOIN dim_contract dc
              ON f.contract_id = dc.contract_id
            JOIN stg_customer_raw s
              ON c.customer_id = s.customerID
            WHERE c.customer_id = :customer_id
        """)

        result = db.execute(
            query,
            {"customer_id": customer_id}
        ).mappings().first()

        if result is None:
            raise HTTPException(
                status_code=404,
                detail=f"Customer '{customer_id}' not found"
            )

        return CustomerResponse(**result)

    finally:
        db.close()

```

96. Test at /docs: look up a known customer ID (e.g. "7590-VHVEG")

```
Curl
curl -X 'GET' \
  'http://127.0.0.1:8000/customer/7590-VMEG' \
  -H 'accept: application/json'

Request URL
http://127.0.0.1:8000/customer/7590-VMEG

Server response

Code    Details
200
Response body
{
  "customer_id": "7590-VMEG",
  "tenure": 1,
  "contract_type": "Month-to-month",
  "internet_service": "DSL",
  "monthly_charges": 29.85,
  "churn": 0
}
Response headers
content-length: 131
content-type: application/json
date: Sat, 08 Aug 2026 13:23:36 GMT
server: uvicorn
```

97. Test a non-existent ID — confirm 404 is returned with a clear message

```
Curl
curl -X 'GET' \
  'http://127.0.0.1:8000/customer/7590' \
  -H 'accept: application/json'

Request URL
http://127.0.0.1:8000/customer/7590

Server response

Code    Details
404      Error: Not Found
Undocumented
Response body
{
  "detail": "Customer '7590' not found"
}
Response headers
content-length: 38
content-type: application/json
date: Sat, 08 Aug 2026 13:24:42 GMT
server: uvicorn
```

LAB API2 — Churn Summary Endpoint

Created a separate file for calculating churn summary. Customer information was retrieved through database queries across the curated tables. The api endpoint is created in the main file

98. Add GET /churn/summary to main.py

99. Create a service layer function get_churn_summary(db) — keep business logic out of the route

100. Query: total customers, total churned, churn rate (as float 0–1)

101. Query: churn rate by contract type (list of {contract_type, churn_rate})

102. Query: churn rate by internet service type

103. Return all of this in a single structured JSON response

Created api2_services.py

```
from sqlalchemy import text

def get_churn_summary(db):
    # 1. Overall churn summary

    overall_query = text("""
```

```

        SELECT
            COUNT(*) AS total_customers,
            SUM(churn) AS total_churned,
            AVG(churn) AS churn_rate
        FROM fact_customer_account
    """)

overall = db.execute(overall_query).mappings().first()

# 2. Churn rate by contract type

contract_query = text("""
    SELECT
        dc.contract_name AS contract_type,
        AVG(f.churn) AS churn_rate
    FROM fact_customer_account f
    JOIN dim_contract dc
        ON f.contract_id = dc.contract_id
    GROUP BY dc.contract_name
    ORDER BY dc.contract_name
""")

contract_results = db.execute(contract_query).mappings().all()

# 3. Churn rate by internet service

internet_query = text("""
    SELECT
        s.InternetService AS internet_service,
        AVG(f.churn) AS churn_rate
    FROM fact_customer_account f
    JOIN stg_customer_raw s
        ON f.customer_id = s.customerID
    GROUP BY s.InternetService
    ORDER BY s.InternetService
""")

internet_results = db.execute(internet_query).mappings().all()

# Return structured result

return {

```

```

    "total_customers": overall["total_customers"],
    "total_churned": overall["total_churned"],
    "churn_rate": float(overall["churn_rate"]),

    "churn_rate_by_contract": [
        {
            "contract_type": row["contract_type"],
            "churn_rate": float(row["churn_rate"])
        }
        for row in contract_results
    ],

    "churn_rate_by_internet_service": [
        {
            "internet_service": row["internet_service"],
            "churn_rate": float(row["churn_rate"])
        }
        for row in internet_results
    ]
}

```

Added route in main

```

@app.get("/churn/summary")
def churn_summary():

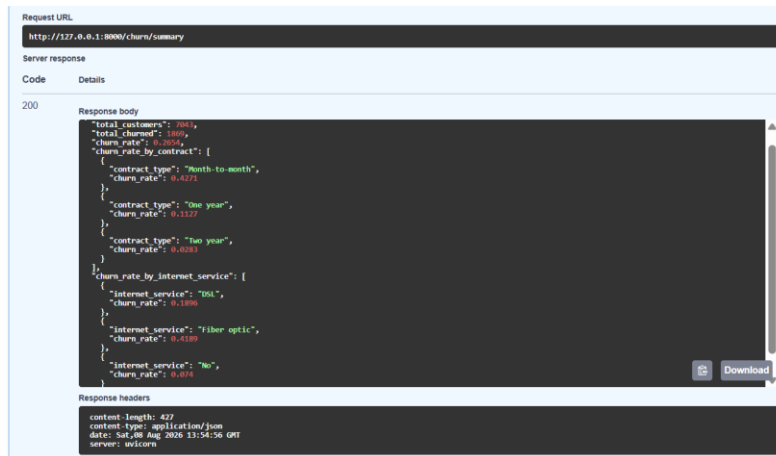
    db = SessionLocal()

    try:
        return get_churn_summary(db)

    finally:
        db.close()

```

104. Test at /docs — verify values match your SQL4 query results



105. Discuss: why create a service layer function rather than putting SQL directly in the route?
- Cleaner routes : endpoints remain small and readable.
 - Reusable logic: another endpoint can call `get_churn_summary()`.
 - Easier testing :we can test the service independently of FastAPI.
 - Easier maintenance: changing churn calculations doesn't require modifying the API route.

LAB API3 — High-Risk Customers Endpoint

Return rule-based flagged customers from SQL6 view. Query parameters are added for limit, min_tenure and max_tenure

106. Add GET /customers/high-risk to main.py
107. Query the v_high_risk_customers view
108. Add optional query parameters: limit (default 50) and min_tenure / max_tenure for filtering
109. Return a list of customer objects with: customer_id, tenure, monthly_charges, contract_type, risk_reason

```
from sqlalchemy import text

def get_high_risk_customers(
    db,
    limit=None,
    min_tenure=None,
    max_tenure=None
):
    query = """
        SELECT
            customer_id,
            tenure,
            monthly_charges,
            contract_name AS contract_type,
```

```

        risk_reason
    FROM v_high_risk_customers
    WHERE 1=1
"""

params = {
    "limit": limit
}

if min_tenure is not None:
    query += " AND tenure >= :min_tenure"
    params["min_tenure"] = min_tenure

if max_tenure is not None:
    query += " AND tenure <= :max_tenure"
    params["max_tenure"] = max_tenure

query += " LIMIT :limit"

result = db.execute(
    text(query),
    params
).mappings().all()

return [
    {
        "customer_id": row["customer_id"],
        "tenure": row["tenure"],
        "monthly_charges": float(row["monthly_charges"]),
        "contract_type": row["contract_type"],
        "risk_reason": row["risk_reason"]
    }
    for row in result
]

```

Added code to main

```

@app.get("/customers/high-risk")
def high_risk_customers(
    limit: int | None = 50,
    min_tenure: int | None = None,
    max_tenure: int | None = None
):
    db = SessionLocal()

```

```

try:
    return get_high_risk_customers(
        db,
        limit=limit,
        min_tenure=min_tenure,
        max_tenure=max_tenure
    )

finally:
    db.close()

```

110. Test with default parameters — confirm results match v_high_risk_customers view

```

curl -X 'GET' \
  'http://127.0.0.1:8080/customers/high-risk' \
  -H 'accept: application/json'

```

Request URL
http://127.0.0.1:8080/customers/high-risk

Server response

Code	Details
200	Response body <pre> { "customer_id": "0004-TLHLJ", "tenure": 4, "monthly_charges": 73.9, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" }, "customer_id": "0013-EXCHZ", "tenure": 3, "monthly_charges": 83.9, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" }, "customer_id": "0013-WI25d", "tenure": 9, "monthly_charges": 69.4, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" }, "customer_id": "0018-NY8OU", "tenure": 5, "monthly_charges": 68.07, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" } </pre>

111. Test with limit=10 and max_tenure=6 — verify filtering works

```

http://127.0.0.1:8080/customers/high-risk?limit=10&max_tenure=6

```

Server response

Code	Details
200	Response body <pre> { "customer_id": "0004-TLHLJ", "tenure": 4, "monthly_charges": 73.9, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" }, "customer_id": "0013-EXCHZ", "tenure": 3, "monthly_charges": 83.9, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" }, "customer_id": "0018-NY8OU", "tenure": 5, "monthly_charges": 68.95, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" }, "customer_id": "0021-IXG6C", "tenure": 1, "monthly_charges": 72.1, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" }, "customer_id": "0021-IXG6C", "tenure": 1, "monthly_charges": 72.1, "contract_type": "Month-to-month", "risk_reason": "Month-to-month contract AND tenure < 12 AND monthly charges > dataset average" } </pre>

Response headers

112. Discuss: this is a rule-based list. How will it change when ML predictions are available (ML5)?

When ML predictions are available, we could get learned patterns instead of fixed business rules. We can predict using many features and get probability score instead of YES/NO

LAB API4 — Customer Features Endpoint

Return the customer_ml_features data for given customer id through GET endpoint

- 113. Add GET /customer/{customer_id}/features to main.py
- 114. Query the customer_ml_features table for the given customer_id
- 115. Return all engineered features: service_count, tenure_bucket, high_charge_flag, is_long_term_customer, has_streaming_bundle, auto_pay_flag, monthly_charges, total_charges
- 116. Raise 404 if customer not found
- 117. Define a Pydantic CustomerFeatures response model

```
class CustomerFeatures(BaseModel):
    customer_id: str
    service_count: int
    tenure_bucket: str
    high_charge_flag: int
    is_long_term_customer: int
    has_streaming_bundle: int
    auto_pay_flag: int
    monthly_charges: float
    total_charges: float

@app.get(
    "/customer/{customer_id}/features",
    response_model=CustomerFeatures
)
def get_customer_features(customer_id: str):

    db = SessionLocal()

    try:
        query = text("""
            SELECT
                customer_id,
                service_count,
                tenure_bucket,
                high_charge_flag,
                is_long_term_customer,
                has_streaming_bundle,
```


120. Add POST /predict-churn to main.py
121. Return a stub response: {"customer_id": input or "N/A", "risk_score": 0.78, "prediction": "Likely to churn", "note": "stub — real model in ML4"}
122. Add a "note" field making it clear this is a stub

```
class ChurnPredictionRequest(BaseModel):
    tenure: int
    monthly_charges: float
    contract_type: str
    service_count: int
@app.post("/predict-churn")
def predict_churn(request: ChurnPredictionRequest):

    return {
        "customer_id": "N/A",
        "risk_score": 0.78,
        "prediction": "Likely to churn",
        "note": "stub — real model in ML4"
    }
```

123. Test via /docs: submit a JSON body and verify the response



124. Verify that submitting wrong types (e.g. tenure as a string) returns a 422 Validation Error automatically from Pydantic



LAB API6 — Add Validation, Logging & Auth

This section focused on enhancing the FastAPI application with validation, security, error handling, and monitoring capabilities. Centralized logging was implemented using FastAPI's

@app.middleware("http") so that every request and response is automatically tracked in one place. API security was implemented using FastAPI's dependency injection mechanism and APIKeyHeader. The API key is read from an environment variable (API_KEY) to avoid hardcoding sensitive credentials.

125. Add field-level Pydantic validators to ChurnPredictionRequest: tenure must be ≥ 0 and ≤ 100 ; monthly_charges must be > 0
126. Add a consistent error response model: ErrorResponse(detail: str, status_code: int)
127. Wrap all DB queries in try/except — return 500 with ErrorResponse if DB is unavailable
128. Add Python logging to every route: log the incoming request (method, path, key params) and the response status
129. Add a 404 handler for all routes using @app.exception_handler(404)

```
from fastapi import FastAPI, HTTPException, Header, Request, Depends
from fastapi.responses import JSONResponse
from fastapi.security import APIKeyHeader
from pydantic import BaseModel, Field, field_validator
from sqlalchemy import create_engine, text
from sqlalchemy.orm import sessionmaker
from sqlalchemy.exc import SQLAlchemyError
import os
import logging

from ap12_services import get_churn_summary
from api3_v_services import get_high_risk_customers

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)
logger = logging.getLogger(__name__)

app = FastAPI(title="Telecom Customer API")

@app.middleware("http")
async def log_requests(request: Request, call_next):
    logger.info(
        "%s %s - incoming request - params=%s",
        request.method,
        request.url.path,
        dict(request.query_params)
    )
```

```

    response = await call_next(request)

    logger.info(
        "%s %s - response_status=%s",
        request.method,
        request.url.path,
        response.status_code
    )

    return response

API_KEY = os.getenv("API_KEY")
if not API_KEY:
    raise RuntimeError("API_KEY environment variable is not set")

api_key_header = APIKeyHeader(
    name="X-API-Key",
    auto_error=False
)

def verify_api_key(
    x_api_key: str | None = Depends(api_key_header)
):
    if x_api_key != API_KEY:
        raise HTTPException(
            status_code=401,
            detail="Invalid or missing API key"
        )

    return x_api_key

@app.exception_handler(404)
async def not_found_handler(request: Request, exc):

    return JSONResponse(
        status_code=404,
        content={
            "detail": "Resource not found",
            "status_code": 404
        }
    )

```

```

class ErrorResponse(BaseModel):
    detail: str
    status_code: int

@app.exception_handler(HTTPException)
async def http_exception_handler(request: Request, exc: HTTPException):

    error = ErrorResponse(
        detail=str(exc.detail),
        status_code=exc.status_code
    )

    return JSONResponse(
        status_code=exc.status_code,
        content=error.model_dump()
    )

DATABASE_URL = "mysql+pymysql://root:root@localhost:3306/telecom_db"

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

class CustomerResponse(BaseModel):
    customer_id: str
    tenure: int
    contract_type: str
    internet_service: str
    monthly_charges: float
    churn: int

@app.get("/customer/{customer_id}", response_model=CustomerResponse ,
dependencies=[Depends(verify_api_key)])
def get_customer(customer_id: str):

    db = SessionLocal()

```

```

try:
    query = text("""
        SELECT
            c.customer_id,
            f.tenure,
            dc.contract_name AS contract_type,
            s.InternetService AS internet_service,
            f.monthly_charges,
            f.churn
        FROM customers c
        JOIN fact_customer_account f
            ON c.customer_id = f.customer_id
        JOIN dim_contract dc
            ON f.contract_id = dc.contract_id
        JOIN stg_customer_raw s
            ON c.customer_id = s.customerID
        WHERE c.customer_id = :customer_id
    """)

    result = db.execute(
        query,
        {"customer_id": customer_id}
    ).mappings().first()

    if result is None:

        raise HTTPException(
            status_code=404,
            detail=f"Customer '{customer_id}' not found"
        )

    return CustomerResponse(**result)
except SQLAlchemyError:
    logger.exception("Database error while fetching customer")

    raise HTTPException(
        status_code=500,
        detail="Database unavailable"
    )

finally:

```

```

        db.close()

@app.get("/churn/summary", dependencies=[Depends(verify_api_key)])
def churn_summary():

    db = SessionLocal()

    try:
        return get_churn_summary(db)
    except SQLAlchemyError:
        logger.exception("Database error while fetching churn summary")

        raise HTTPException(
            status_code=500,
            detail="Database unavailable"
        )

    finally:
        db.close()

@app.get("/customers/high-risk", dependencies=[Depends(verify_api_key)])
def high_risk_customers(
    limit: int | None = 50,
    min_tenure: int | None = None,
    max_tenure: int | None = None
):

    db = SessionLocal()

    try:
        return get_high_risk_customers(
            db,
            limit=limit,
            min_tenure=min_tenure,
            max_tenure=max_tenure
        )
    except SQLAlchemyError:
        logger.exception("Database error while fetching high-risk customers")

        raise HTTPException(
            status_code=500,
            detail="Database unavailable"

```

```

    )

    finally:
        db.close()

class CustomerFeatures(BaseModel):
    customer_id: str
    service_count: int
    tenure_bucket: str
    high_charge_flag: int
    is_long_term_customer: int
    has_streaming_bundle: int
    auto_pay_flag: int
    monthly_charges: float
    total_charges: float

@app.get(
    "/customer/{customer_id}/features",
    response_model=CustomerFeatures,
    dependencies=[Depends(verify_api_key)]
)
def get_customer_features(customer_id: str, ):

    db = SessionLocal()

    try:
        query = text("""
            SELECT
                customer_id,
                service_count,
                tenure_bucket,
                high_charge_flag,
                is_long_term_customer,
                has_streaming_bundle,
                auto_pay_flag,
                monthly_charges,
                total_charges
            FROM customer_ml_features
            WHERE customer_id = :customer_id
        """)

        result = db.execute(

```

```

        query,
        {"customer_id": customer_id}
    ).mappings().first()

    if result is None:
        raise HTTPException(
            status_code=404,
            detail=f"Customer '{customer_id}' features not found"
        )

    return CustomerFeatures(**result)

except SQLAlchemyError:
    logger.exception("Database error while fetching customer features")

    raise HTTPException(
        status_code=500,
        detail="Database unavailable"
    )

finally:
    db.close()

class ChurnPredictionRequest(BaseModel):
    tenure: int
    monthly_charges: float
    contract_type: str
    service_count: int

    @field_validator("tenure")
    @classmethod
    def validate_tenure(cls, value):
        if value < 0 or value > 100:
            raise ValueError("tenure must be between 0 and 100")
        return value

    @field_validator("monthly_charges")
    @classmethod
    def validate_monthly_charges(cls, value):
        if value <= 0:
            raise ValueError("monthly_charges must be greater than 0")
        return value

```

```

@app.post(
    "/predict-churn",
    dependencies=[Depends(verify_api_key)]
)
def predict_churn(request: ChurnPredictionRequest):

    response = {
        "customer_id": "N/A",
        "risk_score": 0.78,
        "prediction": "Likely to churn",
        "note": "stub – real model in ML4"
    }

    return response

```

130. Test: valid request, invalid tenure (negative), unknown customer ID, simulate DB down

```

-H 'Content-Type: application/json' \
-d '{
  "tenure": -2,
  "monthly_charges": -2,
  "contract_type": "string",
  "service_count": 0
}'

```

Request URL

http://127.0.0.1:8000/predict-churn

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```

{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "tenure"
      ],
      "msg": "Value error, tenure must be between 0 and 100",
      "input": -2,
      "ctx": {
        "error": {}
      }
    },
    {
      "type": "value_error",
      "loc": [
        "body",
        "monthly_charges"
      ],
      "msg": "Value error, monthly_charges must be greater than 0",
      "input": -2,
      "ctx": {
        "error": {}
      }
    }
  ]
}

```

```

2026-08-09 20:17:51,692 - INFO - GET /docs - response_status=200
INFO: 127.0.0.1:62801 - "GET /docs HTTP/1.1" 200 OK
2026-08-09 20:17:52,248 - INFO - GET /openapi.json - incoming request - params={}
2026-08-09 20:17:52,295 - INFO - GET /openapi.json - response_status=200
INFO: 127.0.0.1:62801 - "GET /openapi.json HTTP/1.1" 200 OK
2026-08-09 20:24:01,539 - INFO - GET /customers/high-risk - incoming request - params={'limit': '50', 'min_tenure': '-2'}
2026-08-09 20:24:02,000 - INFO - GET /customers/high-risk - response_status=200
INFO: 127.0.0.1:50995 - "GET /customers/high-risk?limit=50&min_tenure=-2 HTTP/1.1" 200 OK
2026-08-09 20:25:28,110 - INFO - POST /predict-churn - incoming request - params={}
2026-08-09 20:25:28,117 - INFO - POST /predict-churn - response_status=422
INFO: 127.0.0.1:50452 - "POST /predict-churn HTTP/1.1" 422 Unprocessable Content

```

← → ↻ http://127.0.0.1:8000/hi

Pretty-print ☐

```

{"detail": "Resource not found", "status_code": 404}

```

Request URL	
<code>http://127.0.0.1:8000/customer/abc/features</code>	
Server response	
Code	Details
404 <i>Undocumented</i>	Error: Not Found
	Response body
	<pre>{ "detail": "Resource not found", "status_code": 404 }</pre>

131. Add a basic token check: read an API_KEY from environment variable; any request without matching X-API-Key header gets 401

Request URL	
<code>http://127.0.0.1:8000/churn/summary</code>	
Server response	
Code	Details
401 <i>Undocumented</i>	Error: Unauthorized
	Response body
	<pre>{ "detail": "Invalid or missing API key", "status_code": 401 }</pre>

132. Discuss: how does this differ from full OAuth2/JWT? What would change in production?

The current API key can handle only simple authentication whereas the OAuth2/JWT can handle both authentication and authorization. API key is sufficient as a basic security layer for this stage. In production, OAuth2/JWT with short-lived tokens, scopes/roles and audit logging would be more appropriate

PHASE 4 - REACT (API CONSUMER)

This section focused on developing the frontend user interface using React and integrating it with the FastAPI backend services. Multiple React pages were created to display customer information churn summary, high risk customers and churn prediction , with data fetched dynamically through FastAPI API endpoints.

LAB RE1 — Setup React App

133. Run `npx create-react-app customer-dashboard`
`npm create vite@latest customer-dashboard`
134. Install axios: `npm install axios`
`cd customer-dashboard`
`npm install`
`npm install axios`
135. Create `src/api.js`: export `BASE_URL` reading from `REACT_APP_API_URL` env variable or defaulting to <http://localhost:8000>

```
import axios from "axios";

export const BASE_URL =
  import.meta.env.VITE_API_URL || "http://localhost:8000";

export const api = axios.create({
  baseURL: BASE_URL,
});
```

136. Add FastAPI CORS middleware: `allow origins=["http://localhost:3000"]` in `main.py`

```
app.add_middleware(
  CORSMiddleware,
  allow_origins=["http://localhost:8000"],
  allow_credentials=True,
  allow_methods=["*"],
  allow_headers=["*"],
)
```

137. In `App.js`, add a `useEffect` that calls `GET /churn/summary` on load and `console.log` the result

```
import { useEffect, useState } from "react";
import { api } from "../api";
```

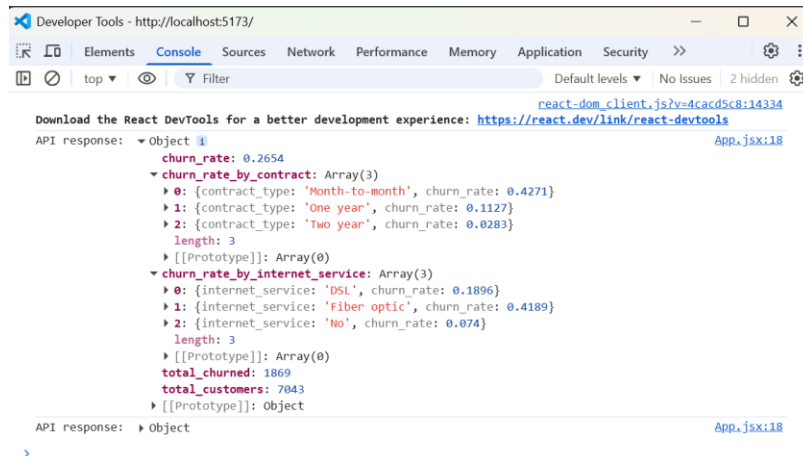
```

function App() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  useEffect(() => {
    const fetchChurnSummary = async () => {
      try {
        const response = await api.get("/churn/summary", {
          headers: {
            "X-API-Key": import.meta.env.VITE_API_KEY,
          },
        });
        console.log("API response:", response.data);
        setData(response.data);
      } catch (err) {
        console.error("API error:", err);
        setError("Failed to load churn summary");
      } finally {
        setLoading(false);
      }
    };
    fetchChurnSummary();
  }, []);
  if (loading) {
    return <h2>Loading...</h2>;
  }
  if (error) {
    return <h2>{error}</h2>;
  }
  return (
    <div>
      <h1>Customer Retention Dashboard</h1>

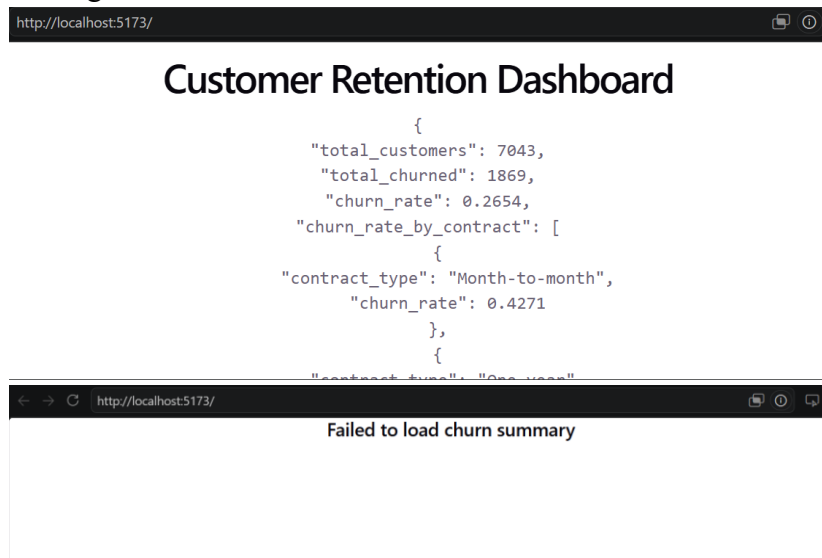
      <pre>
        {JSON.stringify(data, null, 2)}
      </pre>
    </div>
  );
}
export default App;

```

138. Start both servers: `uvicorn main:app --reload` and `npm start`
139. Open browser console — confirm the API response is visible without CORS errors



140. Handle the API call properly: show "Loading..." before data arrives, show error message if call fails



LAB RE2 — Customer Search Page

141. Create src/pages/CustomerSearch.js
142. Add a controlled text input for customer ID (useState for the input value)
143. Add a Search button — onClick triggers the API call
144. Call GET /customer/{id} with the input value using axios
145. Show a loading spinner/message while the request is in flight
146. Display the result in a card layout: customer_id, tenure, contract_type, internet_service, monthly_charges, churn status
147. Handle 404 gracefully: show "Customer not found" instead of a crash
148. Show the churn field with colour coding: red background for churned, green for active
149. Import and add CustomerSearch to App.js with a simple tab or link

```
import { useState } from "react";
import { api } from "../api";

function CustomerSearch() {
  const [customerId, setCustomerId] = useState("");
  const [customer, setCustomer] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  const searchCustomer = async () => {
    if (!customerId.trim()) {
      setError("Please enter a customer ID");
      return;
    }

    setLoading(true);
    setError("");
    setCustomer(null);

    try {
      const response = await api.get(
        `/customer/${customerId.trim()}`,
        {
          headers: {
            "X-API-Key": import.meta.env.VITE_API_KEY,
          },
        }
      );
      setCustomer(response.data);
    } catch (err) {
      if (err.response?.status === 404) {
        setError("Customer not found");
      } else {
        setError("Failed to fetch customer");
      }
    } finally {
      setLoading(false);
    }
  };

  return (
    <div>
```

```

<h2>Customer Search</h2>

<input
  type="text"
  placeholder="Enter Customer ID"
  value={customerId}
  onChange={(e) => setCustomerId(e.target.value)}
/>

<button onClick={searchCustomer}>
  Search
</button>

{loading && <p>Loading...</p>}

{error && <p>{error}</p>}

{customer && (
  <div>
    <h3>Customer Details</h3>

    <p>Customer ID: {customer.customer_id}</p>
    <p>Tenure: {customer.tenure}</p>
    <p>Contract: {customer.contract_type}</p>
    <p>Internet Service: {customer.internet_service}</p>
    <p>Monthly Charges: {customer.monthly_charges}</p>

    <p>
      Churn Status:{" "}
      <span
        style={{
          backgroundColor:
            customer.churn === 1 ? "red" : "green",
          color: "white",
          padding: "5px",
        }}
      >
        {customer.churn === 1 ? "Churned" : "Active"}
      </span>
    </p>
  </div>
)}

```

```

    </div>
  );
}
export default CustomerSearch;

```

```

import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import CustomerSearch from "../pages/CustomerSearch";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/pages/CustomerSearch">
          Customer Search
        </Link>{"|"}
      </nav>
      <Routes>
        <Route
          path="/CustomerSearch"
          element={<CustomerSearch />}
        />
      </Routes>
    </BrowserRouter>
  );
}
export default App;

```

Customer Search

7590-VHVEG Search

Customer Details

Customer ID: 7590-VHVEG
 Tenure: 1
 Contract: Month-to-month
 Internet Service: DSL
 Monthly Charges: 29.85
 Churn Status: Active

[Customer Search](#)

Customer Search

75 Search

Customer not found

LAB RE3 — Churn Summary Page

- 150. Create src/pages/ChurnSummary.js
- 151. On mount (useEffect with []), fetch GET /churn/summary
- 152. Display three metric cards: total customers, total churned, churn rate (as a percentage)
- 153. Display a contract-type churn table: contract_type | churn_rate columns
- 154. Format churn_rate as a percentage: (rate * 100).toFixed(1) + "%"
- 155. Add a simple visual: for each contract type, show a coloured bar whose width is proportional to its churn rate
- 156. Handle loading and error states
- 157. Add a page title and a one-sentence description of what the page shows

```
import { useEffect, useState } from "react";
import { api } from "../api";

function ChurnSummary() {
  const [summary, setSummary] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const fetchChurnSummary = async () => {
      try {
        const response = await api.get("/churn/summary", {
          headers: {
            "X-API-Key": import.meta.env.VITE_API_KEY,
          },
        });

        setSummary(response.data);
      } catch (err) {
        console.error(err);
        setError("Failed to load churn summary");
      } finally {
        setLoading(false);
      }
    };

    fetchChurnSummary();
  }, []);

  if (loading) {
```

```

    return <h2>Loading...</h2>;
  }

  if (error) {
    return <h2>{error}</h2>;
  }

  return (
    <div>
      <h1>Churn Summary</h1>

      <p>
        This page shows overall customer churn and churn rates
        across different contract types.
      </p>

      {/* Metric Cards */}
      <div style={styles.metrics}>
        <div style={styles.card}>
          <h3>Total Customers</h3>
          <p>{summary.total_customers}</p>
        </div>

        <div style={styles.card}>
          <h3>Total Churned</h3>
          <p>{summary.total_churned}</p>
        </div>

        <div style={styles.card}>
          <h3>Churn Rate</h3>
          <p>
            {(summary.churn_rate * 100).toFixed(1)}%
          </p>
        </div>
      </div>

      {/* Contract Churn Table */}
      <h2>Churn Rate by Contract Type</h2>

      <table border="1" cellPadding="10">
        <thead>
          <tr>

```

```

        <th>Contract Type</th>
        <th>Churn Rate</th>
        <th>Visual</th>
      </tr>
    </thead>

    <tbody>
      {summary.churn_rate_by_contract.map((item) => (
        <tr key={item.contract_type}>
          <td>{item.contract_type}</td>

          <td>
            {(item.churn_rate * 100).toFixed(1)}%
          </td>

          <td>
            <div style={styles.barContainer}>
              <div
                style={{
                  ...styles.bar,
                  width: `${item.churn_rate * 100}%`,
                }}
              />
            </div>
          </td>
        </tr>
      )})
    </tbody>
  </table>
</div>
);
}

const styles = {
  metrics: {
    display: "flex",
    gap: "20px",
    margin: "20px 0",
  },

  card: {
    border: "1px solid #ddd",

```

```

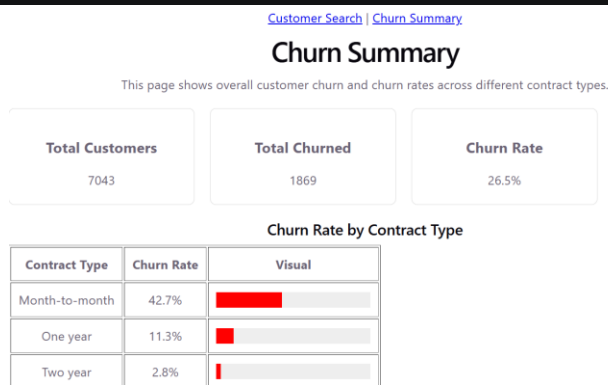
borderRadius: "8px",
padding: "20px",
width: "200px",
},

barContainer: {
width: "200px",
height: "20px",
backgroundColor: "#eee",
},

bar: {
height: "20px",
backgroundColor: "red",
},
};

export default ChurnSummary;

```



LAB RE4 — High-Risk Customers Page

158. Create src/pages/HighRiskCustomers.js
159. On mount, fetch GET /customers/high-risk (default limit 50)
160. Render results in an HTML table: customer_id | tenure | monthly_charges | contract_type | risk_reason
161. Show a count banner: "X high-risk customers identified"
162. Add a "Load More" button that refetches with limit increased by 50
163. Add a sort toggle on the tenure column: clicking the header sorts ascending/descending
164. Format monthly_charges as currency: "\$" + value.toFixed(2)

165. Handle the case where 0 results are returned: show "No high-risk customers found"

```
import { useEffect, useState } from "react";
import { api } from "../api";

function HighRiskCustomers() {
  const [customers, setCustomers] = useState([]);
  const [limit, setLimit] = useState(50);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const [sortAscending, setSortAscending] = useState(true);

  const fetchHighRiskCustomers = async (newLimit) => {
    try {
      setLoading(true);
      setError("");

      const response = await api.get("/customers/high-risk", {
        params: {
          limit: newLimit,
        },
        headers: {
          "X-API-Key": import.meta.env.VITE_API_KEY,
        },
      });

      setCustomers(response.data);
    } catch (err) {
      console.error(err);
      setError("Failed to load high-risk customers");
    } finally {
      setLoading(false);
    }
  };

  // Fetch first 50 when page loads
  useEffect(() => {
    fetchHighRiskCustomers(50);
  }, []);

  // Load 50 more
  const loadMore = () => {
    const newLimit = limit + 50;
```

```

    setLimit(newLimit);
    fetchHighRiskCustomers(newLimit);
  };

  // Sort by tenure
  const sortedCustomers = [...customers].sort((a, b) => {
    if (sortAscending) {
      return a.tenure - b.tenure;
    }

    return b.tenure - a.tenure;
  });

  const toggleSort = () => {
    setSortAscending(!sortAscending);
  };

  if (loading) {
    return <h2>Loading...</h2>;
  }

  if (error) {
    return <h2>{error}</h2>;
  }

  return (
    <div>
      <h1>High-Risk Customers</h1>

      <p>
        Customers identified as high-risk using explainable
        business rules.
      </p>

      { /* Count Banner */ }
      <div>
        {customers.length} high-risk customers identified
      </div>

      { /* No Results */ }
    </div>
  );

```

```

{customers.length === 0 ? (
  <p>No high-risk customers found</p>
) : (
  <>
    <table border="1" cellPadding="10">
      <thead>
        <tr>
          <th>Customer ID</th>

          <th onClick={toggleSort} style={{ cursor: "pointer" }}>
            Tenure {sortAscending ? "↑" : "↓"}
          </th>

          <th>Monthly Charges</th>
          <th>Contract Type</th>
          <th>Risk Reason</th>
        </tr>
      </thead>

      <tbody>
        {sortedCustomers.map((customer) => (
          <tr key={customer.customer_id}>
            <td>{customer.customer_id}</td>

            <td>{customer.tenure}</td>

            <td>
              ${customer.monthly_charges.toFixed(2)}
            </td>

            <td>{customer.contract_type}</td>

            <td>{customer.risk_reason}</td>
          </tr>
        ))}
      </tbody>
    </table>

    { /* Load More */ }
    <button onClick={loadMore}>
      Load More
    </button>
  </>
)

```

```

    </>
  })
</div>
);
}
export default HighRiskCustomers;

```

[Customer Search](#) | [Churn Summary](#) | [High Risk Customers](#)

High-Risk Customers

Customers identified as high-risk using explainable business rules.
50 high-risk customers identified

Customer ID	Tenure ↓	Monthly Charges	Contract Type	Risk Reason
0094-OIFMO	11	\$95.00	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset average
0315-LVCRK	11	\$84.80	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset average
0195-IESCP	10	\$85.25	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset average
0013-MHZWF	9	\$69.40	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset average
0196-JTUQI	9	\$75.20	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset average
1062-LHZOD	1	\$69.90	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset average
1069-XAIEM	1	\$85.05	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset average
1084-UQCHV	1	\$69.75	Month-to-month	Month-to-month contract AND tenure < 12 AND monthly charges > dataset average

[Load More](#)

LAB RE5 — Churn Prediction Page

166. Create src/pages/ChurnPrediction.js
167. Build a form with 4 controlled inputs: tenure (number), monthly_charges (number), contract_type (select: Month-to-month / One year / Two year), service_count (number 0–6)
168. On submit, POST to /predict-churn with the form values as a JSON body
169. Display the result: risk_score as a percentage, prediction text, confidence level
170. Add visual risk indicator: green for low risk (<0.3), amber for medium (0.3–0.6), red for high (>0.6)
171. Show a loading state on the submit button while the request is in flight
172. Handle validation errors (422): display the Pydantic error message to the user
173. Test with stub response first; retest after ML4 wires up the real model
174. Full end-to-end test: enter values → POST → model response → coloured risk display

```
import { useState } from "react";
```

```

import { api } from "../api";

function ChurnPrediction() {
  const [form, setForm] = useState({
    tenure: "",
    monthly_charges: "",
    contract_type: "Month-to-month",
    service_count: "",
  });
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm({
      ...form,
      [name]: value,
    });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError("");
    setResult(null);
    try {
      const response = await api.post(
        "/predict-churn",
        {
          tenure: Number(form.tenure),
          monthly_charges: Number(form.monthly_charges),
          contract_type: form.contract_type,
          service_count: Number(form.service_count),
        },
        {
          headers: {
            "X-API-Key": import.meta.env.VITE_API_KEY,
          },
        }
      );
      setResult(response.data);
    } catch (err) {
      console.error(err);
    }
  };
}

```

```

// Pydantic 422 validation error
if (err.response?.status === 422) {
  const validationErrors = err.response.data.detail;
  if (Array.isArray(validationErrors)) {
    setError(
      validationErrors
        .map((item) => item.msg)
        .join(", ")
    );
  } else {
    setError("Validation error");
  }
} else {
  setError("Failed to get churn prediction");
}
} finally {
  setLoading(false);
}
};

// Determine risk level
const getRiskLevel = (score) => {
  if (score < 0.3) {
    return {
      text: "Low Risk",
      color: "green",
    };
  }
  if (score <= 0.6) {
    return {
      text: "Medium Risk",
      color: "orange",
    };
  }
  return {
    text: "High Risk",
    color: "red",
  };
};

return (
  <div>
    <h1>Churn Prediction</h1>
    <p>
      Enter customer information to predict the likelihood

```

of churn.

```
</p>
<form onSubmit={handleSubmit}>
  {/* Tenure */}
  <div>
    <label>Tenure (months)</label>
    <input
      type="number"
      name="tenure"
      value={form.tenure}
      onChange={handleChange}
    />
  </div>
  {/* Monthly Charges */}
  <div>
    <label>Monthly Charges</label>
    <input
      type="number"
      step="0.01"
      name="monthly_charges"
      value={form.monthly_charges}
      onChange={handleChange}
    />
  </div>
  {/* Contract Type */}
  <div>
    <label>Contract Type</label>
    <select
      name="contract_type"
      value={form.contract_type}
      onChange={handleChange}
    >
      <option value="Month-to-month">
        Month-to-month
      </option>
      <option value="One year">
        One year
      </option>
      <option value="Two year">
        Two year
      </option>
    </select>
  </div>
```

```

    { /* Service Count */}
    <div>
      <label>Service Count</label>
      <input
        type="number"
        name="service_count"
        min="0"
        max="6"
        value={form.service_count}
        onChange={handleChange}
      />
    </div>
    <button type="submit" disabled={loading}>
      {loading ? "Predicting..." : "Predict Churn"}
    </button>
  </form>
  { /* Error */}
  {error && (
    <div>
      <p>{error}</p>
    </div>
  )}
  { /* Result */}
  {result && (
    <PredictionResult result={result} getRiskLevel={getRiskLevel} />
  )}
</div>
);
}
function PredictionResult({ result, getRiskLevel }) {
  const risk = getRiskLevel(result.risk_score);
  return (
    <div
      style={{
        marginTop: "30px",
        padding: "20px",
        border: `3px solid ${risk.color}`,
        borderRadius: "8px",
      }}
    >
      <h2>Prediction Result</h2>
      <h3
        style={{

```

```

        color: risk.color,
      }}
    >
    {risk.text}
  </h3>
  <p>
    <strong>Risk Score:</strong>{" "}
    {(result.risk_score * 100).toFixed(1)}%
  </p>
  <p>
    <strong>Prediction:</strong>{" "}
    {result.prediction}
  </p>
  <p>
    <strong>Confidence Level:</strong>{" "}
    {(result.confidence * 100).toFixed(1)}%

  </p>
  {result.note && (
    <p>
      <strong>Note:</strong> {result.note}
    </p>
  )}
</div>
);
}
export default ChurnPrediction;

```

Churn Prediction

Enter customer information to predict the likelihood of churn.

Tenure (months)

Monthly Charges

Contract Type

Service Count

Predict Churn

Prediction Result

High Risk

Risk Score: 78.0%
Prediction: Likely to churn
Confidence Level: Medium
Note: stub — real model in ML4

PHASE 5 - DATA ENGINEERING

This dataset is best suited for a batch-oriented data engineering workflow, as it consists of periodic customer data extracts rather than real-time event streams. The end-to-end pipeline was designed to process data through multiple stages, beginning with the raw CSV ingestion, followed by loading into a staging layer, data cleaning and preprocessing, feature engineering, and PySpark-based analytical processing.

LAB DE1 — Design Customer Data Pipeline Architecture

175. Draw the pipeline layers: CRM Extract → Raw File → Staging Table → Cleaned Table → Feature Table → ML Scoring → Dashboard/API

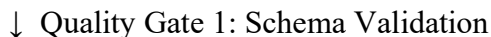
CRM Daily Extract



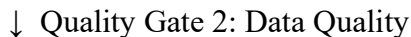
Raw CSV



Staging Table



Cleaned Table



Feature Table



PySpark Analytics



ML Scoring



Dashboard / API

176. For each layer, define: what data format, what tool handles it, who reads it next

CRM Extract Layer

Format: Database records

Tool: CRM/SQL

Read by: Ingestion

Raw File Layer

Format: CSV

Tool: Python / filesystem

Read by: Staging loader

Staging Layer

Format: SQL table

Tool: MySQL

Read by: Cleaning

Cleaned Layer

Format: SQL/DataFrame

Tool: Python/PySpark

Read by: Feature engineering

Features Layer

Format: SQL table

Tool: Python/PySpark

Read by: ML

ML Scoring Layer

Format: Predictions/table

Tool: ML model

Read by: API/Dashboard

Serving Layer

Format: JSON/API response

Tool: FastAPI

Read by: Frontend/users

177. Identify the two key quality gates: schema validation (before staging load) and data quality check (before feature build)

Gate 1 — Schema validation

Before loading CSV into staging:

- Required columns exist
- Column names are correct
- Expected data types are valid
- File is not empty
- No unexpected columns
- Required customer ID exists

Gate 2 — Data quality

Before feature engineering:

- No NULLs in important columns
- customer_id is unique
- monthly_charges ≥ 0
- tenure ≥ 0
- churn contains only 0/1
- Expected row count is reasonable

178. Define the incremental processing question: what happens when the same customer appears in two consecutive daily extracts?

Today's record should update customer rather than create a duplicate.

179. Map each layer to the labs that implement it: CP5 handles layers 1–3, DE2 formalises ingestion, DE3 formalises cleaning, etc.

CP5 Lab: Raw data → staging → initial SQL tables

DE1 Lab: Architecture/design

DE2 Lab: Ingestion

DE3 Lab: Cleaning

DE4 Lab: Feature engineering

DE5 Lab: PySpark analytics

DE7 Lab: Airflow orchestration

180. Produce a one-page architecture document: layer diagram + tool mapping + quality gate definitions

LAB DE2 — Build the Ingestion Flow

181. Create the folder structure: data/landing/, data/raw/, data/rejected/, data/logs/

182. Place customer_churn.csv into data/landing/

183. Write detect_files() — scans landing/ and returns list of CSV files

184. Write validate_schema(filepath) — reads the header row and checks it matches EXPECTED_COLS (21 column names exactly)

185. Write load_to_staging(filepath, engine) — loads valid CSV into stg_customer_raw, returns row count

186. Write log_ingestion(filename, status, rows, reason) — writes to ingestion_log table

187. Write process_landing() — orchestrates: detect → validate → load → log for each file

188. Test: run with a valid file (expect LOADED), an invalid file missing a column (expect REJECTED)
189. Verify: SELECT * FROM ingestion_log shows both entries

```
import os
import csv
import shutil
import pandas as pd
from sqlalchemy import create_engine, text
from datetime import datetime

EXPECTED_COLS = [
    "customerID", "gender", "SeniorCitizen", "Partner", "Dependents",
    "tenure", "PhoneService", "MultipleLines", "InternetService",
    "OnlineSecurity", "OnlineBackup", "DeviceProtection", "TechSupport",
    "StreamingTV", "StreamingMovies", "Contract", "PaperlessBilling",
    "PaymentMethod", "MonthlyCharges", "TotalCharges", "Churn"
]

LANDING_DIR = "data/landing"
RAW_DIR = "data/raw"
REJECTED_DIR = "data/rejected"
LOG_DIR = "data/logs"

def detect_files():
    """Return all CSV files present in landing/."""

    files = []

    for filename in os.listdir(LANDING_DIR):
        if filename.lower().endswith(".csv"):
            files.append(os.path.join(LANDING_DIR, filename))

    return files

def validate_schema(filepath):
    """Check whether CSV header exactly matches EXPECTED_COLS."""

    with open(filepath, newline="", encoding="utf-8") as f:
        header = next(csv.reader(f))

    return header == EXPECTED_COLS, header
```

```

def load_to_staging(filepath, engine):
    """Load valid CSV into staging table."""

    df = pd.read_csv(filepath)

    df.to_sql(
        "stg_customer_raw",
        engine,
        if_exists="replace",
        index=False
    )
    return len(df)

def log_ingestion(filename, status, rows, reason, engine):
    """Insert ingestion result into ingestion_log."""

    log_data = pd.DataFrame([
        {
            "filename": filename,
            "status": status,
            "rows": rows,
            "reason": reason,
            "ingested_at": datetime.now()
        }
    ])

    log_data.to_sql(
        "ingestion_log",
        engine,
        if_exists="append",
        index=False
    )

def process_landing(engine):
    """Detect, validate, load and log each landing file."""

    files = detect_files()

    for filepath in files:

        filename = os.path.basename(filepath)

        # 1. Validate schema

```

```
valid, header = validate_schema(filepath)

if not valid:

    print(f"REJECTED: {filename}")

    log_ingestion(
        filename=filename,
        status="REJECTED",
        rows=0,
        reason="Schema validation failed",
        engine=engine
    )

    shutil.move(
        filepath,
        os.path.join(REJECTED_DIR, filename)
    )

    continue

# 2. Load into staging
try:
    rows = load_to_staging(filepath, engine)

    print(f"LOADED: {filename} ({rows} rows)")

    log_ingestion(
        filename=filename,
        status="LOADED",
        rows=rows,
        reason="Schema valid and loaded successfully",
        engine=engine
    )

    # Move successfully processed file
    shutil.move(
        filepath,
        os.path.join(RAW_DIR, filename)
    )
```

```

except Exception as e:

    print(f"ERROR: {filename} -> {e}")

    log_ingestion(
        filename=filename,
        status="ERROR",
        rows=0,
        reason=str(e),
        engine=engine
    )

if __name__ == "__main__":

    # Change these according to your MySQL setup
    engine = create_engine(
        "mysql+pymysql://root:root@localhost/telecom_db_de"
    )
    # Create required folders
    os.makedirs(LANDING_DIR, exist_ok=True)
    os.makedirs(RAW_DIR, exist_ok=True)
    os.makedirs(REJECTED_DIR, exist_ok=True)
    os.makedirs(LOG_DIR, exist_ok=True)
    process_landing(engine)

```

```

PS C:\Users\yuvasree.r\Desktop\python test> & C:\Users\yuvasree.r\Desktop\python test\de2_ingestion.py
"LOADED: customer_churn.csv (7043 rows)
REJECTED: old.csv"

```

```

mysql> select * from ingestion_log;

```

filename	status	rows	reason	ingested_at
customer_churn.csv	LOADED	7043	Schema valid and loaded successfully	2026-08-14 12:13:29
old.csv	REJECTED	0	Schema validation failed	2026-08-14 12:13:29

```

2 rows in set (0.00 sec)

```

LAB DE3 — Build the Cleaning Stage

190. Write `clean_staging(engine)` — reads `stg_customer_raw` into a `DataFrame`, runs `CustomerCleaner.clean()`, writes to a `cleaned_customers` table

```

def clean_staging(engine):
    # Read staging table
    df = pd.read_sql(
        "SELECT * FROM stg_customer_raw",
        engine
    )

```

```

raw_count = len(df)
print(f"Raw staging rows: {raw_count}")
cleaner = CustomerCleaner(df)
cleaned_df = cleaner.clean()
cleaned_count = len(cleaned_df)
print(f"Cleaned rows: {cleaned_count}")

# Row count check
assert cleaned_count == raw_count, (
    f"Row count mismatch: "
    f"staging={raw_count}, cleaned={cleaned_count}"
)
print(
    f"PASS - Row count check: "
    f"{cleaned_count}/{raw_count}"
)

# Data quality checks

null_customer_id = cleaned_df["customer_i_d"].isna().sum()
if null_customer_id == 0:
    print("PASS - customer_id NULL check: 0")
else:
    print(
        f"FAIL - customer_id NULL check: "
        f"{null_customer_id}"
    )
    raise AssertionError("customer_id contains NULLs")

null_monthly = cleaned_df["monthly_charges"].isna().sum()
if null_monthly == 0:
    print("PASS - monthly_charges NULL check: 0")
else:
    print(
        f"FAIL - monthly_charges NULL check: "
        f"{null_monthly}"
    )
    raise AssertionError("monthly_charges contains NULLs")

invalid_churn = (
    ~cleaned_df["churn"].isin([0, 1])
).sum()

```

```

if invalid_churn == 0:
    print("PASS - churn values check: 0 invalid")
else:
    print(
        f"FAIL - churn values check: "
        f"{invalid_churn} invalid"
    )
    raise AssertionError("Invalid churn values")

# Write cleaned table
cleaned_df.to_sql(
    "cleaned_customers",
    engine,
    if_exists="replace",
    index=False
)
print(
    f"cleaned_customers populated: "
    f"{cleaned_count} rows"
)
return cleaned_df

```

191. Write `build_curated_tables(engine)` — populates `dim_contract`, `dim_payment`, `customers`, `fact_customer_account` from the cleaned staging data

```

def build_curated_tables(engine):
    df = pd.read_sql(
        "SELECT * FROM cleaned_customers",
        engine
    )
    # dim_contract
    dim_contract = (
        df[["contract"]]
        .drop_duplicates()
        .reset_index(drop=True)
    )
    dim_contract.insert(
        0,
        "contract_i_d",
        range(1, len(dim_contract) + 1)
    )
    dim_contract.to_sql(
        "dim_contract",
        engine,

```

```

        if_exists="replace",
        index=False
    )
    # dim_payment
    dim_payment = (
        df[["payment_method"]]
        .drop_duplicates()
        .reset_index(drop=True)
    )
    dim_payment.insert(
        0,
        "payment_i_d",
        range(1, len(dim_payment) + 1)
    )
    dim_payment.to_sql(
        "dim_payment",
        engine,
        if_exists="replace",
        index=False
    )
    # customers
    customers = df[
        [
            "customer_i_d",
            "gender",
            "senior_citizen",
            "partner",
            "dependents"
        ]
    ].copy()
    customers.to_sql(
        "customers",
        engine,
        if_exists="replace",
        index=False
    )
    # fact_customer_account
    df = df.merge(
        dim_contract,
        on="contract",
        how="left"
    )
    # Map payment → payment_id

```

```

df = df.merge(
    dim_payment,
    on="payment_method",
    how="left"
)
fact_customer_account = df[
    [
        "customer_i_d",
        "contract_i_d",
        "payment_i_d",
        "tenure",
        "monthly_charges",
        "total_charges",
        "churn"
    ]
].copy()

fact_customer_account.to_sql(
    "fact_customer_account",
    engine,
    if_exists="replace",
    index=False
)
print("PASS - dim_contract populated")
print("PASS - dim_payment populated")
print("PASS - customers populated")
print("PASS - fact_customer_account populated")

```

192. Add a row count check: assert cleaned_customers row count == stg_customer_raw row count (no rows dropped unless intentional)
193. Add data quality assertions: no NULLs in customer_id or monthly_charges; churn column values are only 0 or 1
194. Write a simple quality report: for each check, print PASS or FAIL with count
195. Test: run clean_staging() followed by build_curated_tables() — verify all four curated tables are populated

```

Raw staging rows: 7043
2026-08-14 13:09:14,993 - INFO - Starting customer data cleaning.
2026-08-14 13:09:14,994 - INFO - Column names standardized to snake_case.
2026-08-14 13:09:15,002 - INFO - Converted total_charges to float. 11 blank values converted to NaN.
2026-08-14 13:09:15,009 - INFO - Binary columns normalized.
2026-08-14 13:09:15,010 - INFO - Filled 11 missing total_charges values using monthly_charges.
2026-08-14 13:09:15,010 - INFO - Customer data cleaning completed.
Cleaned rows: 7043
PASS - Row count check: 7043/7043
PASS - customer_id NULL check: 0
PASS - monthly_charges NULL check: 0
PASS - churn values check: 0 invalid
cleaned_customers populated: 7043 rows
PASS - dim_contract populated
PASS - dim_payment populated
PASS - customers populated
PASS - fact_customer_account populated

```

```

mysql> select * from dim_contract;
+-----+-----+
| contract_i_d | contract |
+-----+-----+
| 1 | Month-to-month |
| 2 | One year |
| 3 | Two year |
+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> select * from dim_payment;
+-----+-----+
| payment_i_d | payment_method |
+-----+-----+
| 1 | Electronic check |
| 2 | Mailed check |
| 3 | Bank transfer (automatic) |
| 4 | Credit card (automatic) |
+-----+-----+
4 rows in set (0.00 sec)

```

```

mysql> select count(*) from fact_customer_account;
+-----+
| count(*) |
+-----+
| 7043 |
+-----+
1 row in set (0.01 sec)

```

```

mysql> select count(*) from customers;
+-----+
| count(*) |
+-----+
| 7043 |
+-----+
1 row in set (0.01 sec)

```

LAB DE4 — Build the Feature Pipeline

196. Write `build_features(engine)` — reads `cleaned_customers`, computes all 6 features from CP4, writes to `customer_ml_features` table
197. Ensure the feature logic exactly matches CP4: same thresholds (median for `high_charge_flag`), same column names, same data types
198. Add a validation step: count of `customer_ml_features` should equal count of `cleaned_customers`
199. Add a schema check: confirm all 6 feature columns are present and non-null

```
import pandas as pd
from sqlalchemy import create_engine

from cp4_feat_engg import build_features

def build_ml_features(engine):

    df = pd.read_sql(
        "SELECT * FROM cleaned_customers",
        engine
    )
    cleaned_count = len(df)
    print(f"Cleaned customers: {cleaned_count}")

    df = build_features(df)
    original_columns = [
        "customer_i_d",
        "gender",
        "senior_citizen",
        "partner",
        "dependents",
        "tenure",
        "phone_service",
        "multiple_lines",
        "internet_service",
        "online_security",
        "online_backup",
        "device_protection",
        "tech_support",
        "streaming_t_v",
        "streaming_movies",
        "contract",
        "paperless_billing",
        "payment_method",
        "monthly_charges",
        "total_charges",
        "churn"
    ]
    engineered_features = [
        "tenure_bucket",
        "high_charge_flag",
        "service_count",
        "is_long_term_customer",
```

```

        "has_streaming_bundle",
        "auto_pay_flag"
    ]

    required_columns = (
        original_columns +
        engineered_features
    )
    missing_columns = [
        col
        for col in required_columns
        if col not in df.columns
    ]

    if missing_columns:
        raise AssertionError(
            f"Missing columns: {missing_columns}"
        )

    print("\nPASS - All 27 columns are present")

    ml_features = df[
        required_columns
    ].copy()
    # Integer columns
    integer_columns = [
        "senior_citizen",
        "partner",
        "dependents",
        "tenure",
        "phone_service",
        "online_security",
        "online_backup",
        "device_protection",
        "tech_support",
        "streaming_t_v",
        "streaming_movies",
        "paperless_billing",
        "churn",
        "high_charge_flag",
        "service_count",
        "is_long_term_customer",
        "has_streaming_bundle",
    ]

```

```

    "auto_pay_flag"
]

for col in integer_columns:
    ml_features[col] = ml_features[col].astype(int)

# Float columns
ml_features["monthly_charges"] = (
    ml_features["monthly_charges"].astype(float)
)

ml_features["total_charges"] = (
    ml_features["total_charges"].astype(float)
)

print("\nSchema / NULL Checks")
print("-----")

for col in engineered_features:
    null_count = ml_features[col].isna().sum()
    if null_count == 0:
        print(
            f"PASS - {col}: 0 NULLs"
        )
    else:
        print(
            f"FAIL - {col}: "
            f"{null_count} NULLs"
        )

        raise AssertionError(
            f"{col} contains NULL values"
        )

null_customer_id = (
    ml_features["customer_id"].isna().sum()
)

null_monthly_charges = (
    ml_features["monthly_charges"].isna().sum()
)

```

```

invalid_churn = (
    ~ml_features["churn"].isin([0, 1])
).sum()

if null_customer_id == 0:
    print("PASS - customer_id NULL check: 0")
else:
    print(
        f"FAIL - customer_id NULL check: "
        f"{null_customer_id}"
    )
    raise AssertionError(
        "customer_id contains NULLs"
    )
if null_monthly_charges == 0:
    print(
        "PASS - monthly_charges NULL check: 0"
    )
else:
    print(
        f"FAIL - monthly_charges NULL check: "
        f"{null_monthly_charges}"
    )
    raise AssertionError(
        "monthly_charges contains NULLs"
    )

if invalid_churn == 0:
    print(
        "PASS - churn values check: 0 invalid"
    )
else:
    print(
        f"FAIL - churn values check: "
        f"{invalid_churn} invalid"
    )
    raise AssertionError(
        "Invalid churn values"
    )

feature_count = len(ml_features)
if feature_count == cleaned_count:
    print(

```

```

        f"PASS - Row count: "
        f"{feature_count}/{cleaned_count}"
    )
else:
    print(
        f"FAIL - Row count: "
        f"{feature_count}/{cleaned_count}"
    )
    raise AssertionError(
        "Row count mismatch"
    )
ml_features.to_sql(
    "customer_ml_features",
    engine,
    if_exists="replace",
    index=False
)
print("\nSUCCESS")
print(
    "customer_ml_features populated with "
    f"{len(ml_features)} rows and "
    f"{len(ml_features.columns)} columns."
)

return ml_features
if __name__ == "__main__":
    engine = create_engine(
        "mysql+pymysql://root:root@localhost/telecom_db_de"
    )
    build_ml_features(engine)

```

200. Test: run the function and query `customer_ml_features` — spot-check 3 customers against the CSV output from CP4

```
mysql> select * from customer_ml_features limit 3;
```

customer_id	gender	senior_citizen	partner	dependents	tenure	phone_service	multiple_lines	internet_service	online_security	online_backup	device_protection	tech_support	streaming_tv	streaming_movies	contract	paperless_billing	payment_method	monthly_charges	total_charges	churn	tenure_bucket	high_charge_flag	service_count	is_long_term_customer	has_streaming_bundle	auto_pay_flag
7590-VHVEG	Female	0	1	0	1	0	0	No phone service	DSL	0	0	Month-to-month	1	0	0	0	Electronic check	29.85	29.85	0	0-12	0	0	0	0	
5575-GNVDE	Male	0	0	0	34	0	1	No	DSL	0	0	One year	2	0	0	0	Mailed check	56.95	1889.5	0	25-48	0	0	0	0	
3668-QPYBK	Male	1	0	0	2	0	1	No	DSL	0	0	Month-to-month	2	0	0	0	Mailed check	53.85	108.15	1	0-12	0	0	0	0	

- The customer_ml_features table contains the same rows as the csv file from CP4
201. Discuss: if the CP4 feature definitions change, what do you need to update in the pipeline?

If the features change then I need to update the build_features function from CP4, the feature_columns and data type handling in DE4 also needs to be modified

LAB DE5 — Process Customer Data with PySpark

202. Create a SparkSession: local[2], 4g driver memory (7K rows is tiny — this config is intentionally minimal)

```
from pyspark.sql import SparkSession
from pyspark.sql.types import (
    StructType,
    StructField,
    StringType,
    IntegerType,
    DoubleType
)
from pyspark.sql import functions as F

import os
os.environ["HADOOP_HOME"] = r"C:\Users\yuvaree.r\Desktop\python test\hadoop"
os.environ["PATH"] = os.environ["PATH"] + r";C:\Users\yuvaree.r\Desktop\python test\hadoop\bin"

spark = (
    SparkSession.builder
        .appName("CustomerChurnDE5")
        .master("local[2]")
```

```

.config("spark.driver.memory", "4g")
.getOrCreate()
)

```

203. Define a manual schema (StructType) for the CSV — all correct types including StringType for TotalCharges

```

schema = StructType([

    StructField("customerID", StringType(), True),
    StructField("gender", StringType(), True),
    StructField("SeniorCitizen", IntegerType(), True),
    StructField("Partner", StringType(), True),
    StructField("Dependents", StringType(), True),
    StructField("tenure", IntegerType(), True),
    StructField("PhoneService", StringType(), True),
    StructField("MultipleLines", StringType(), True),
    StructField("InternetService", StringType(), True),
    StructField("OnlineSecurity", StringType(), True),
    StructField("OnlineBackup", StringType(), True),
    StructField("DeviceProtection", StringType(), True),
    StructField("TechSupport", StringType(), True),
    StructField("StreamingTV", StringType(), True),
    StructField("StreamingMovies", StringType(), True),
    StructField("Contract", StringType(), True),
    StructField("PaperlessBilling", StringType(), True),
    StructField("PaymentMethod", StringType(), True),
    StructField("MonthlyCharges", DoubleType(), True),
    StructField("TotalCharges", StringType(), True),
    StructField("Churn", StringType(), True)

])

```

204. Read customer_churn.csv with the defined schema — print printSchema() and compare to pandas dtypes

```

df = (
    spark.read
    .option("header", True)
    .schema(schema)
    .csv("customer_churn.csv")
)

print("\n===== ORIGINAL SPARK SCHEMA =====")
df.printSchema()

```

```
print("\n===== SAMPLE DATA =====")
df.show(5, truncate=False)
```

```
===== ORIGINAL SPARK SCHEMA =====
root
|-- customerID: string (nullable = true)
|-- gender: string (nullable = true)
|-- SeniorCitizen: integer (nullable = true)
|-- Partner: string (nullable = true)
|-- Dependents: string (nullable = true)
|-- tenure: integer (nullable = true)
|-- PhoneService: string (nullable = true)
|-- MultipleLines: string (nullable = true)
|-- InternetService: string (nullable = true)
|-- OnlineSecurity: string (nullable = true)
|-- OnlineBackup: string (nullable = true)
|-- DeviceProtection: string (nullable = true)
|-- TechSupport: string (nullable = true)
|-- StreamingTV: string (nullable = true)
|-- StreamingMovies: string (nullable = true)
|-- Contract: string (nullable = true)
|-- PaperlessBilling: string (nullable = true)
|-- PaymentMethod: string (nullable = true)
|-- MonthlyCharges: double (nullable = true)
|-- TotalCharges: string (nullable = true)
|-- Churn: string (nullable = true)
```

205. Fix TotalCharges blanks: use `F.regexp_replace()` to replace blank strings, then cast to `DoubleType`

```
df = df.withColumn(
    "TotalCharges",
    F.regexp_replace(
        F.col("TotalCharges"),
        r"^\s*$",
        ""
    )
)
df = df.withColumn(
    "TotalCharges",
    F.when(
        F.col("TotalCharges") == "",
        None
    ).otherwise(
        F.col("TotalCharges")
    )
)
df = df.withColumn(
    "TotalCharges",
    F.col("TotalCharges").cast(DoubleType())
)
```

206. Encode the Churn column: use `F.when(F.col("Churn")== "Yes", 1).otherwise(0)` — Spark equivalent of `map()`

```
df = df.withColumn(
    "churn_encoded",
    F.when(
        F.col("Churn") == "Yes",
        1
    ).otherwise(0)
)

print("\n===== CLEANED SPARK SCHEMA =====")
df.printSchema()
```

207. GroupBy Contract: compute `COUNT(*)` as `customer_count` and `AVG(churn_encoded)` as `churn_rate` — show results

```
contract_summary = (
    df.groupBy("Contract")
    .agg(
        F.count("*").alias("customer_count"),
        F.avg("churn_encoded").alias("churn_rate")
    )
    .orderBy("Contract")
)

print("\n===== CHURN BY CONTRACT =====")
contract_summary.show(truncate=False)
```

```
===== CHURN BY CONTRACT =====
+-----+-----+-----+
|Contract|customer_count|churn_rate|
+-----+-----+-----+
|Month-to-month|3875|0.4270967741935484|
|One year|1473|0.11269517990495587|
|Two year|1695|0.02831858407079646|
+-----+-----+-----+
```

208. GroupBy InternetService: compute `AVG(MonthlyCharges)` and `AVG(churn_encoded)` — show results

```
internet_summary = (
```

```
df.groupBy("InternetService")
  .agg(
    F.avg("MonthlyCharges").alias("avg_monthly_charges"),
    F.avg("churn_encoded").alias("churn_rate")
  )
  .orderBy("InternetService")
)
print("\n===== INTERNET SERVICE SUMMARY =====")
internet_summary.show(truncate=False)
```

```
===== INTERNET SERVICE SUMMARY =====
+-----+-----+-----+
|InternetService|avg_monthly_charges|churn_rate|
+-----+-----+-----+
|DSL            |58.10216852540261  |0.1895910780669145|
|Fiber optic    |91.50012919896615  |0.4189276485788114|
|No             |21.079193971166454 |0.07404980340760157|
+-----+-----+-----+
```

209. Cross-validate: Spark churn rates must exactly match your CP3 pandas groupby results
The spark churn rates match the CP3 results

```
===== Churn Rate by Contract =====
contract
Month-to-month    42.709677
One year          11.269518
Two year          2.831858
Name: churn, dtype: float64

Highest Churn Contract: Month-to-month

===== Churn Rate by Internet Service =====
internet_service
DSL              18.959108
Fiber optic      41.892765
No               7.404980
Name: churn, dtype: float64
```

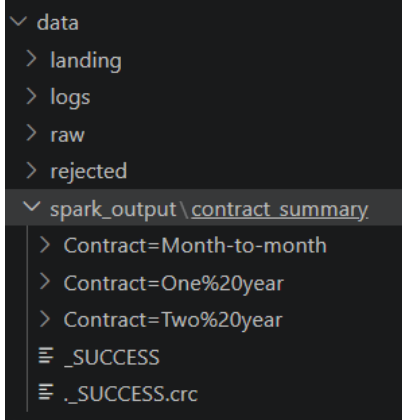
210. Write the contract-level summary as Parquet: data/spark_output/contract_summary/
partitioned by Contract

```
output_path = "data/spark_output/contract_summary/"

(
  contract_summary
  .write
  .mode("overwrite")
  .partitionBy("Contract")
  .parquet(output_path)
)
```

```
)

print(f"\nParquet written to: {output_path}")
```



211. Read the Parquet back, print schema and count — confirm round-trip is clean

```
parquet_df = spark.read.parquet(output_path)
print("\n===== PARQUET SCHEMA =====")
parquet_df.printSchema()

print("\n===== PARQUET DATA =====")
parquet_df.show(truncate=False)

# Round-trip validation

original_count = contract_summary.count()
parquet_count = parquet_df.count()

print("\n===== ROUND-TRIP VALIDATION =====")

print(f"Original summary rows : {original_count}")
print(f"Parquet summary rows : {parquet_count}")

assert original_count == parquet_count
print("PASS - Parquet row count matches")

spark.stop()
```

```

===== PARQUET SCHEMA =====
root
|-- customer_count: long (nullable = true)
|-- churn_rate: double (nullable = true)
|-- Contract: string (nullable = true)

===== PARQUET DATA =====
+-----+-----+-----+
|customer_count|churn_rate      |Contract      |
+-----+-----+-----+
|1473          |0.11269517990495587|One year      |
|1695          |0.02831858407079646|Two year      |
|3875          |0.4270967741935484 |Month-to-month|
+-----+-----+-----+

===== ROUND-TRIP VALIDATION =====
Original summary rows : 3
Parquet summary rows  : 3
PASS - Parquet row count matches

```

LAB DE6 — Design Incremental Processing

212. Discuss the three cases: new customer (INSERT), updated customer (UPDATE), unchanged (skip)
213. Implement upsert logic for the customers table using SQLAlchemy: INSERT ... ON CONFLICT (customer_id) DO UPDATE

```

import pandas as pd
from sqlalchemy import create_engine, text
from datetime import datetime

def get_existing_customers(engine):
    query = """
        SELECT
            customer_i_d,
            gender,
            senior_citizen,
            partner,
            dependents
        FROM customers
    """
    return pd.read_sql(query, engine)

def classify_records(incoming_df, existing_df):
    # Make customer IDs consistent
    incoming_df["customer_i_d"] = (
        incoming_df["customer_i_d"]

```

```

        .astype(str)
        .str.strip()
    )

    existing_df["customer_i_d"] = (
        existing_df["customer_i_d"]
        .astype(str)
        .str.strip()
    )
    # Existing customer IDs
    existing_ids = set(
        existing_df["customer_i_d"]
    )
    # Create lookup dictionary
    existing_dict = (
        existing_df
        .set_index("customer_i_d")
        .to_dict("index")
    )

    rows_inserted = 0
    rows_updated = 0
    rows_unchanged = 0

    records_to_upsert = []

    compare_columns = [
        "gender",
        "senior_citizen",
        "partner",
        "dependents"
    ]

    for _, row in incoming_df.iterrows():
        customer_id = row["customer_i_d"]
        if customer_id not in existing_ids:
            rows_inserted += 1
            records_to_upsert.append(
                row.to_dict()
            )
        else:

```

```

        old_values = existing_dict[customer_id]
        changed = False
        for col in compare_columns:
            old_value = old_values[col]
            new_value = row[col]
            # Handle NULLs
            if pd.isna(old_value) and pd.isna(new_value):
                continue
            if pd.isna(old_value) != pd.isna(new_value):
                changed = True
                break
            # Compare as strings
            if str(old_value).strip() != str(new_value).strip():
                changed = True
                break

        if changed:
            rows_updated += 1
            records_to_upsert.append(
                row.to_dict()
            )

        else:
            rows_unchanged += 1
    return (
        records_to_upsert,
        rows_inserted,
        rows_updated,
        rows_unchanged
    )
def upsert_customers(records, engine):
    if not records:
        return
    query = text("""
        INSERT INTO customers (
            customer_i_d,
            gender,
            senior_citizen,
            partner,
            dependents
        )
        VALUES (
            :customer_i_d,

```

```

        :gender,
        :senior_citizen,
        :partner,
        :dependents
    )

    ON DUPLICATE KEY UPDATE
        gender = VALUES(gender),
        senior_citizen = VALUES(senior_citizen),
        partner = VALUES(partner),
        dependents = VALUES(dependents)
    """

with engine.begin() as connection:
    for record in records:
        connection.execute(
            query,
            {
                "customer_i_d": record["customer_i_d"],
                "gender": record["gender"],
                "senior_citizen": int(
                    record["senior_citizen"]
                ),
                "partner": int(
                    record["partner"]
                ),
                "dependents": int(
                    record["dependents"]
                )
            }
        )

def create_pipeline_log_table(engine):
    query = text("""
        CREATE TABLE IF NOT EXISTS pipeline_run_log (
            id INT AUTO_INCREMENT PRIMARY KEY,
            file_name VARCHAR(255),
            rows_inserted INT,
            rows_updated INT,
            rows_unchanged INT,
            run_timestamp DATETIME
        )
    """)

```

```

        with engine.begin() as connection:
            connection.execute(query)

def log_pipeline_run(
    engine,
    file_name,
    rows_inserted,
    rows_updated,
    rows_unchanged
):
    query = text("""
        INSERT INTO pipeline_run_log (
            file_name,
            rows_inserted,
            rows_updated,
            rows_unchanged,
            run_timestamp
        )
        VALUES (
            :file_name,
            :rows_inserted,
            :rows_updated,
            :rows_unchanged,
            :run_timestamp
        )
    """)

    with engine.begin() as connection:
        connection.execute(
            query,
            {
                "file_name": file_name,
                "rows_inserted": rows_inserted,
                "rows_updated": rows_updated,
                "rows_unchanged": rows_unchanged,
                "run_timestamp": datetime.now()
            }
        )

def process_incremental(filepath, engine):

    print("\n=====")
    print("INCREMENTAL CUSTOMER PROCESSING")

```

```

print("=====")
incoming_df = pd.read_csv(filepath)
print(
    f"Incoming CSV rows: {len(incoming_df)}"
)

existing_df = get_existing_customers(engine)
print(
    f"Existing customers: {len(existing_df)}"
)
(
    records,
    rows_inserted,
    rows_updated,
    rows_unchanged
) = classify_records(
    incoming_df,
    existing_df
)
upsert_customers(
    records,
    engine
)

file_name = filepath.split("/")[-1]
log_pipeline_run(
    engine,
    file_name,
    rows_inserted,
    rows_updated,
    rows_unchanged
)

print("\nPIPELINE RESULT")
print("-----")
print(f"Rows inserted      : {rows_inserted}")
print(f"Rows updated       : {rows_updated}")
print(f"Rows unchanged     : {rows_unchanged}")

return (
    rows_inserted,
    rows_updated,
    rows_unchanged
)

```

```

)

if __name__ == "__main__":
    engine = create_engine(
        "mysql+pymysql://root:root@localhost/telecom_db_de"
    )
    # Create log table if it doesn't exist
    create_pipeline_log_table(engine)
    # Process daily customer extract
    process_incremental(
        "new_data.csv",
        engine
    )

```

214. Test Case 1 — new customer: add one row to the CSV with a new customer_id, re-run pipeline, verify count increases by 1

```

=====
INCREMENTAL CUSTOMER PROCESSING
=====
Incoming CSV rows: 1
Existing customers: 7043

PIPELINE RESULT
-----
Rows inserted   : 1
Rows updated    : 0
Rows unchanged  : 0

mysql> select count(*) from customers;
+-----+
| count(*) |
+-----+
|      7044 |
+-----+
1 row in set (0.01 sec)

```

215. Test Case 2 — updated customer: change tenure of an existing customer_id in the CSV, re-run, verify the row is updated not duplicated

```

=====
INCREMENTAL CUSTOMER PROCESSING
=====
Incoming CSV rows: 1
Existing customers: 7044

PIPELINE RESULT
-----
Rows inserted   : 0
Rows updated    : 1
Rows unchanged  : 0

```

216. Test Case 3 — same file re-run: re-run with the original CSV, verify count stays at 7043 with no duplicates

```

c:\Users\yuvasree.r\Desktop\python test\de6_incre.py:343: SyntaxWarning: invalid
"output\customer_clean_20260731_120356.csv",

=====
INCREMENTAL CUSTOMER PROCESSING
=====
Incoming CSV rows: 7043
Existing customers: 7043

PIPELINE RESULT
-----
Rows inserted   : 0
Rows updated    : 0
Rows unchanged  : 7043

```

217. Add a pipeline run log: record file_name, rows_inserted, rows_updated, rows_unchanged, run_timestamp

```

mysql> select * from pipeline_run_log;

```

id	file_name	rows_inserted	rows_updated	rows_unchanged	run_timestamp
1	output\customer_clean_20260731_120356.csv	0	0	7043	2026-08-15 16:31:20
2	new_data.csv	2	0	0	2026-08-15 16:43:12
3	new_data.csv	2	0	0	2026-08-15 16:45:58
4	new_data.csv	2	0	0	2026-08-15 16:54:28
5	new_data.csv	2	0	1	2026-08-15 17:01:24
6	new_data.csv	2	0	1	2026-08-15 19:00:09
7	new_data.csv	2	0	1	2026-08-15 19:08:18
8	new_data.csv	1	0	0	2026-08-15 19:19:39
9	new_data.csv	0	1	0	2026-08-15 19:21:11
10	new_data.csv	0	0	1	2026-08-15 19:22:15

218. Discuss: why is idempotency important in production pipelines?
Idempotency makes sure that reprocessing the same data multiple times does not change the final outcome, preventing duplicates and maintaining data consistency in production pipelines

LAB DE7 — Add Data Quality Checks

219. Define 6 quality checks as functions: check_null_rate(df, col, threshold), check_value_range(df, col, min, max), check_allowed_values(df, col, allowed_set), check_row_count(df, expected_min), check_no_duplicates(df, key_col), check_churn_distribution(df)
220. Run null rate check: monthly_charges and tenure must have 0% nulls
221. Run value range check: tenure must be 0–100; monthly_charges must be > 0
222. Run allowed values check: contract must be one of {Month-to-month, One year, Two year}
223. Run row count check: must have at least 7000 rows (catches truncated files)
224. Run duplicate check: no duplicate customer_id values
225. Produce a quality_report.json with: check name, PASS/FAIL, value found, threshold
226. If any CRITICAL check fails: raise an exception to stop the pipeline
227. If WARNING checks fail: log warning but continue

```

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
import json
import os

```

```

from datetime import datetime

import os
os.environ["HADOOP_HOME"] = r"C:\Users\yuvasree.r\Desktop\python test\hadoop"
os.environ["PATH"] = os.environ["PATH"] + r";C:\Users\yuvasree.r\Desktop\python test\hadoop\bin"

spark = (
    SparkSession.builder
        .appName("CustomerDataQuality")
        .master("local[2]")
        .config("spark.driver.memory", "4g")
        .config("spark.jars", r"C:\Users\yuvasree.r\Desktop\python test\mysql-connector-
j-26.7.0\mysql-connector-j-26.7.0.jar")
        .getOrCreate()
)

# Quality Check 1 - NULL RATE
def check_null_rate(df, col, threshold):
    total_rows = df.count()
    null_count = df.filter(F.col(col).isNull()).count()
    if total_rows == 0:
        null_rate = 100
    else:
        null_rate = (null_count / total_rows) * 100
    passed = null_rate <= threshold
    return {
        "check_name": f"null_rate_{col}",
        "status": "PASS" if passed else "FAIL",
        "value_found": round(null_rate, 4),
        "threshold": threshold,
        "severity": "CRITICAL"
    }

# Quality Check 2 - VALUE RANGE
def check_value_range(df, col, min_value, max_value=None):
    min_found = df.select(F.min(col)).first()[0]
    max_found = df.select(F.max(col)).first()[0]
    if max_value is None:
        passed = min_found is not None and min_found > min_value
        threshold = f"> {min_value}"
    else:
        passed = min_found is not None and min_found >= min_value and max_found <=
max_value
        threshold = f"{min_value} - {max_value}"

```

```

    return {
        "check_name": f"value_range_{col}",
        "status": "PASS" if passed else "FAIL",
        "value_found": {
            "min": min_found,
            "max": max_found
        },
        "threshold": threshold,
        "severity": "CRITICAL"
    }

# Quality Check 3 - ALLOWED VALUES
def check_allowed_values(df, col, allowed_set):
    invalid_count = df.filter(~F.col(col).isin(list(allowed_set))).count()
    passed = invalid_count == 0
    return {
        "check_name": f"allowed_values_{col}",
        "status": "PASS" if passed else "FAIL",
        "value_found": invalid_count,
        "threshold": list(allowed_set),
        "severity": "CRITICAL"
    }

# Quality Check 4 - ROW COUNT
def check_row_count(df, expected_min):
    actual_count = df.count()
    passed = actual_count >= expected_min
    return {
        "check_name": "row_count",
        "status": "PASS" if passed else "FAIL",
        "value_found": actual_count,
        "threshold": expected_min,
        "severity": "CRITICAL"
    }

# Quality Check 5 - DUPLICATES
def check_no_duplicates(df, key_col):
    total_count = df.count()
    unique_count = df.select(key_col).distinct().count()
    duplicate_count = total_count - unique_count
    passed = duplicate_count == 0
    return {
        "check_name": f"duplicates_{key_col}",
        "status": "PASS" if passed else "FAIL",

```

```

        "value_found": duplicate_count,
        "threshold": 0,
        "severity": "CRITICAL"
    }

# Quality Check 6 - CHURN DISTRIBUTION
def check_churn_distribution(df):
    total_count = df.count()
    churn_count = df.filter(F.col("churn") == 1).count()
    churn_rate = churn_count / total_count if total_count > 0 else 0
    churn_percentage = churn_rate * 100
    # Warning if churn is outside 5%-95%
    passed = 5 <= churn_percentage <= 95
    return {
        "check_name": "churn_distribution",
        "status": "PASS" if passed else "FAIL",
        "value_found": round(churn_percentage, 2),
        "threshold": "5% - 95%",
        "severity": "WARNING"
    }

# RUN ALL CHECKS
def run_quality_checks(df):
    results = []
    # NULL: monthly_charges
    results.append(check_null_rate(df, "monthly_charges", 0))
    # NULL: tenure
    results.append(check_null_rate(df, "tenure", 0))
    # tenure: 0-100
    results.append(check_value_range(df, "tenure", 0, 100))
    # monthly_charges > 0
    results.append(check_value_range(df, "monthly_charges", 0))
    # contract allowed values
    results.append(check_allowed_values(df, "contract", {"Month-to-month", "One
year", "Two year"}))
    # at least 7000 rows
    results.append(check_row_count(df, 7000))
    # duplicate customer_i_d
    results.append(check_no_duplicates(df, "customer_i_d"))
    # churn distribution
    results.append(check_churn_distribution(df))
    return results

# 9. PRINT REPORT
def print_quality_report(results):

```

```

print("\n=====")
print("DATA QUALITY REPORT")
print("=====")
for result in results:
    print(f"{result['check_name']:35}{result['status']:7}{result['severity']}")
    print(f"   Value      : {result['value_found']}")
    print(f"   Threshold : {result['threshold']}")

# 10. SAVE JSON REPORT
def save_quality_report(results):
    os.makedirs("data/logs", exist_ok=True)
    report = {"run_timestamp": datetime.now().isoformat(), "checks": results}
    with open("data/logs/quality_report.json", "w") as file:
        json.dump(report, file, indent=4, default=str)
    print("\nReport saved to " "data/logs/quality_report.json")

# 11. HANDLE FAILURES
def handle_quality_results(results):
    critical_failures = [
        r for r in results
        if r["status"] == "FAIL"
        and r["severity"] == "CRITICAL"
    ]
    warnings = [
        r for r in results
        if r["status"] == "FAIL"
        and r["severity"] == "WARNING"
    ]
    for warning in warnings:
        print(f"WARNING: " f"{warning['check_name']} failed")
    # Critical failures
    if critical_failures:
        print("\nCRITICAL QUALITY FAILURE")
        for failure in critical_failures:
            print(f"FAIL: {failure['check_name']}")
            raise Exception(
                "Pipeline stopped because "
                "critical quality checks failed."
            )
        print("\nAll critical checks passed.")

# 12. MAIN
if __name__ == "__main__":

    # Read cleaned table

```

```
df = (spark.read.format("jdbc")
      .option("url", "jdbc:mysql://localhost:3306/telecom_db_de")
      .option("dbtable", "cleaned_customers")
      .option("user", "root")
      .option("password", "root")
      .option("driver", "com.mysql.cj.jdbc.Driver")
      .load())
```

```
print("\nCleaned customer data:")
df.show(5)
```

```
print("\nSchema:")
df.printSchema()
```

```
results = run_quality_checks(df)
print_quality_report(results)
save_quality_report(results)
handle_quality_results(results)
spark.stop()
```

```
Cleaned customer data:
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|customer_id|gender|senior_citizen|partner|dependents|tenure|phone_service|multiple_lines|internet_service|online_security|online_backup|device_protection|tech_support|streaming_tv|streaming_movies|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|7590-VHVEG|Female|0|1|0|1|0|No phone service|0|DSL|No|Yes|No|No|No| |
|No|Month-to-month|0|1|Electronic check|29.85|29.85|0|DSL|Yes|No|Yes|No|No|No|
|5575-GNVOE|Male|0|0|0|34|1|56.95|1889.5|0|DSL|Yes|No|Yes|No|No|
|No|One year|0|0|Mailed check|56.95|1889.5|0|DSL|Yes|Yes|Yes|No|No|No|
|3668-QPYBK|Male|0|0|0|2|1|53.85|108.15|1|DSL|Yes|Yes|No|No|No|
|No|Month-to-month|1|Mailed check|53.85|108.15|1|DSL|Yes|No|Yes|Yes|Yes|No|
|7795-CFOCW|Male|0|0|0|45|0|No phone service|0|DSL|Yes|No|Yes|Yes|No|
|No|One year|0|Bank transfer (au...|42.3|1840.75|0|Fiber optic|No|No|No|No|No|
|9237-HQITU|Female|0|0|0|2|1|70.7|151.65|1|Fiber optic|No|No|No|No|No|
|No|Month-to-month|1|Electronic check|70.7|151.65|1|Fiber optic|No|No|No|No|No|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

Schema:
root
 |-- customer_id: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- senior_citizen: long (nullable = true)
 |-- partner: long (nullable = true)
 |-- dependents: long (nullable = true)
 |-- tenure: long (nullable = true)
 |-- phone_service: long (nullable = true)
 |-- multiple_lines: string (nullable = true)
 |-- internet_service: string (nullable = true)
 |-- online_security: string (nullable = true)
 |-- online_backup: string (nullable = true)
 |-- device_protection: string (nullable = true)
 |-- tech_support: string (nullable = true)
 |-- streaming_tv: string (nullable = true)
 |-- streaming_movies: string (nullable = true)
 |-- contract: string (nullable = true)
 |-- paperless_billing: long (nullable = true)
 |-- payment_method: string (nullable = true)
 |-- monthly_charges: double (nullable = true)
 |-- total_charges: double (nullable = true)
 |-- churn: long (nullable = true)
```

```

=====
DATA QUALITY REPORT
=====
null_rate_monthly_charges      PASS    CRITICAL
Value      : 0.0
Threshold  : 0
null_rate_tenure              PASS    CRITICAL
Value      : 0.0
Threshold  : 0
value_range_tenure            PASS    CRITICAL
Value      : {'min': 0, 'max': 72}
Threshold  : 0 - 100
value_range_monthly_charges   PASS    CRITICAL
Value      : {'min': 18.25, 'max': 118.75}
Threshold  : > 0
allowed_values_contract       PASS    CRITICAL
Value      : 0
Threshold  : ['Month-to-month', 'Two year', 'One year']
row_count                    PASS    CRITICAL
Value      : 7043
Threshold  : 7000
duplicates_customer_i_d      PASS    CRITICAL
Value      : 0
Threshold  : 0
churn_distribution            PASS    WARNING
Value      : 26.54
Threshold  : 5% - 95%

Report saved to data/logs/quality_report.json

All critical checks passed.

```

```

re.py  de7_quality.py  {} quality_report.json  new_data.csv  {} settings.json
logs > {} quality_report.json > ...
{
  "run_timestamp": "2026-08-15T20:09:31.165962",
  "checks": [
    {
      "check_name": "null_rate_monthly_charges",
      "status": "PASS",
      "value_found": 0.0,
      "threshold": 0,
      "severity": "CRITICAL"
    },
    {
      "check_name": "null_rate_tenure",
      "status": "PASS",
      "value_found": 0.0,
      "threshold": 0,
      "severity": "CRITICAL"
    },
    {
      "check_name": "value_range_tenure",
      "status": "PASS",
      "value_found": {
        "min": 0,
        "max": 72
      },
      "threshold": "0 - 100",
      "severity": "CRITICAL"
    },
    {
      "check_name": "value_range_monthly_charges",
      "status": "PASS",
      "value_found": {
        "min": 18.25,
        "max": 118.75
      },
      "threshold": "> 0",
      "severity": "CRITICAL"
    },
    {
      "check_name": "allowed_values_contract",
      "status": "PASS",
      "value_found": 0
    }
  ]
}

```

PHASE 6 - MACHINE LEARNING

LAB ML1 — Define the ML Problem

This section focused on preparing the engineered dataset (CP4 CSV) for machine learning model training.

237. Load customer_ml_features (from SQL5 or CP4 CSV)
238. Identify the target column: churn (already encoded as 0/1)
239. Identify the drop column: customer_id (identifier, not a feature)
240. List all numeric columns: tenure, monthly_charges, total_charges, service_count, high_charge_flag, is_long_term_customer, auto_pay_flag, has_streaming_bundle
241. List the remaining categorical columns: contract_type, internet_service — these still need encoding
242. Apply one-hot encoding to contract_type and internet_service using pd.get_dummies()
243. Print the final feature set X and target y — print shapes

```
Final feature set:
<bound method NDFrame.head of
No      1      29.85      29.85      1 ...      False      True      False      False
1      34      56.95      1889.50      2 ...      False      True      False      False
2       2      53.85      108.15      2 ...      False      True      False      False
3      45      42.30      1840.75      3 ...      False      True      False      False
4       2      70.70      151.65      0 ...      False      False      True      False
...     ...     ...     ...     ...     ...     ...     ...     ...
7038    24      84.80      1990.50      5 ...      False      True      False      False
7039    72     103.20      7362.90      4 ...      False      False      True      False
7040    11      29.60      346.45      1 ...      False      True      False      False
7041     4       74.40      306.60      0 ...      False      False      True      False
7042    66     105.65      6844.50      5 ...       True      False      True      False

[7043 rows x 14 columns]>

Target:
<bound method NDFrame.head of 0
1      0
2      1
3      0
4      1
..
7038    0
7039    0
7040    0
7041    1
7042    0
Name: churn, Length: 7043, dtype: int64>

X shape: (7043, 14)
y shape: (7043,)
```

244. Print the class balance: y.value_counts(normalize=True) — note the ~73%/27% split

```
Class balance:
churn
0    0.73463
1    0.26537
Name: proportion, dtype: float64

Class counts:
churn
0    5174
1    1869
Name: count, dtype: int64
```

```
import pandas as pd

df = pd.read_csv(r"output\customer_features_20260731_120356.csv")
print("Original shape:", df.shape)
print(df.columns.tolist())
```

```

y = df["churn"]

numeric_cols = [
    "tenure",
    "monthly_charges",
    "total_charges",
    "service_count",
    "high_charge_flag",
    "is_long_term_customer",
    "has_streaming_bundle",
    "auto_pay_flag" ]
categorical_cols = ["contract", "internet_service"]
X = df[numeric_cols + categorical_cols].copy()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=False)
print("\nFinal feature set:")
print(X.head)

print("\nTarget:")
print(y.head)

print("\nX shape:", X.shape)
print("y shape:", y.shape)

print("\nClass balance:")
print(y.value_counts(normalize=True))

print("\nClass counts:")
print(y.value_counts())

```

245. Discuss: is this imbalance a problem? What metric should we NOT use as primary?
(accuracy — predict all 0 = 73% accuracy trivially)

The imbalance is moderate and should be considered. Accuracy is not a reliable primary metric. F1-score or Recall are more appropriate.

LAB ML2 — Build and Compare Baseline Models

Logistic regression and decision tree are used to create models to predict churn . Since decision tree provides better performance metrics it is chosen as final model

246. Split X and y: train_test_split(test_size=0.2, stratify=y, random_state=42)

247. Train a LogisticRegression(max_iter=1000) and a DecisionTreeClassifier(max_depth=5)
248. Predict on the test set for both models
249. For each model: print classification_report with target_names=["Active", "Churned"]
250. For each model: print confusion_matrix

```
=====
LOGISTIC REGRESSION
=====
Classification Report:
      precision    recall  f1-score   support

   Active       0.82       0.89       0.85       1035
   Churned       0.60       0.47       0.53        374

 accuracy       0.78       0.78       0.78       1409
 macro avg       0.71       0.68       0.69       1409
 weighted avg    0.76       0.78       0.77       1409

Confusion Matrix:
[[918 117]
 [197 177]]

=====
DECISION TREE
=====
Classification Report:
      precision    recall  f1-score   support

   Active       0.86       0.85       0.86       1035
   Churned       0.60       0.63       0.62        374

 accuracy       0.79       0.79       0.79       1409
 macro avg       0.73       0.74       0.74       1409
 weighted avg    0.79       0.79       0.79       1409

Confusion Matrix:
[[878 157]
 [138 236]]
```

251. Compute 5-fold cross-validation F1 score for both: cross_val_score(scoring="f1")

```
5-Fold CV F1 Scores
Logistic Regression: [0.57771261 0.59281437 0.54247392 0.54954955 0.56193353]
Logistic Mean CV F1: 0.5648967970088024
Decision Tree: [0.62531646 0.5304878 0.55084746 0.56818182 0.58036984]
Decision Tree Mean CV F1: 0.5710406759821852
```

252. Create a comparison table: Model | Accuracy | Precision (Churned) | Recall (Churned) | F1 (Churned) | CV F1

```
=====
MODEL COMPARISON
=====
      Model Accuracy Precision (Churned) Recall (Churned) F1 (Churned) CV F1
0 Logistic Regression    0.7771          0.6020          0.4733    0.5299 0.5649
1      Decision Tree    0.7906          0.6005          0.6310    0.6154 0.5710
```

253. Choose the better model based on F1 for the Churned class and justify the choice
Decision Tree was selected as the better model because it achieved a higher F1-score (0.6154 vs 0.5299). It also achieved higher recall (0.6310 vs 0.4733), meaning it missed fewer actual churners. The Decision Tree also had a higher 5-fold cross-validation F1 score (0.5710 vs 0.5648), indicating better generalization. Therefore, the Decision Tree with max_depth=5 is preferred for this churn prediction task.
254. Save both models with joblib: models/logistic_churn.pkl and models/tree_churn.pkl

```
▼ models
  ≡ logistic_churn.pkl
  ≡ tree_churn.pkl
```

```
import os
import joblib

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (classification_report, confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
                                                    random_state=42)

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)

logistic_model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
tree_model = DecisionTreeClassifier(max_depth=5, random_state=42)

logistic_model.fit(X_train, y_train)
tree_model.fit(X_train, y_train)

logistic_pred = logistic_model.predict(X_test)
tree_pred = tree_model.predict(X_test)

print("\n" + "=" * 60)
print("LOGISTIC REGRESSION")
print("=" * 60)
print("\nClassification Report:")
print(classification_report(y_test, logistic_pred, target_names=["Active",
                                                                    "Churned"]))
print("Confusion Matrix:")
print(confusion_matrix(y_test, logistic_pred))

print("\n" + "=" * 60)
print("DECISION TREE")
```

```

print("=" * 60)
print("\nClassification Report:")
print(classification_report(y_test, tree_pred, target_names=["Active", "Churned"]))
print("Confusion Matrix:")
print(confusion_matrix(y_test, tree_pred))

logistic_cv_f1 = cross_val_score(logistic_model, X, y, cv=5, scoring="f1")
tree_cv_f1 = cross_val_score(tree_model, X, y, cv=5, scoring="f1")

print("\n5-Fold CV F1 Scores")
print("Logistic Regression:", logistic_cv_f1)
print("Logistic Mean CV F1:", logistic_cv_f1.mean())
print("Decision Tree:", tree_cv_f1)
print("Decision Tree Mean CV F1:", tree_cv_f1.mean())

comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree"],
    "Accuracy": [accuracy_score(y_test, logistic_pred), accuracy_score(y_test,
tree_pred)],
    "Precision (Churned)": [precision_score(y_test, logistic_pred),
precision_score(y_test, tree_pred)],
    "Recall (Churned)": [recall_score(y_test, logistic_pred), recall_score(y_test,
tree_pred)],
    "F1 (Churned)": [f1_score(y_test, logistic_pred), f1_score(y_test, tree_pred)],
    "CV F1": [logistic_cv_f1.mean(), tree_cv_f1.mean()]
})

print("\n" + "=" * 80)
print("MODEL COMPARISON")
print("=" * 80)
print(comparison.round(4))

logistic_f1 = f1_score(y_test, logistic_pred)
tree_f1 = f1_score(y_test, tree_pred)

if logistic_f1 > tree_f1:
    print("\nBetter model: Logistic Regression")
    print("Logistic Regression has the higher F1 score " "for the Churned class.")
else:
    print("\nBetter model: Decision Tree")
    print("Decision Tree has the higher F1 score " "for the Churned class.")

```

```

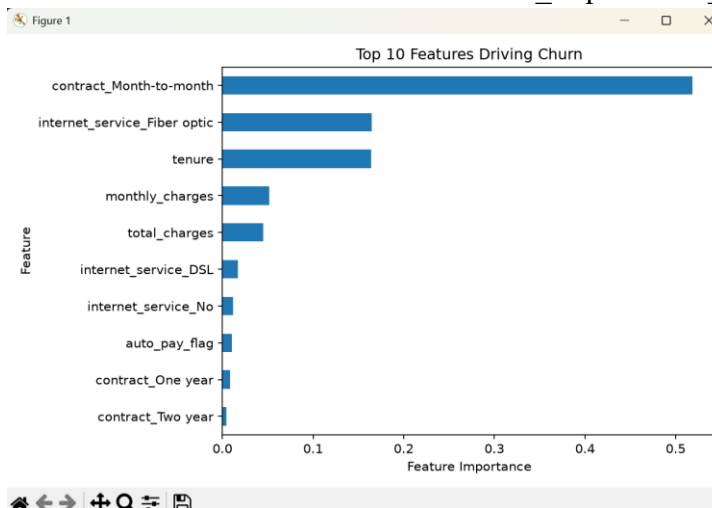
os.makedirs("models", exist_ok=True)
joblib.dump(logistic_model, "models/logistic_churn.pkl")
joblib.dump(tree_model, "models/tree_churn.pkl")
print("\nModels saved successfully:")
print("models/logistic_churn.pkl")
print("models/tree_churn.pkl")

```

LAB ML3 — Interpret Model Behaviour

The important features from model are identified . The high risk customers are predicted.

255. Load the best model from ML2 (Logistic Regression: inspect coefficients; Decision Tree: inspect feature importances)
256. For Logistic Regression: extract `coef_` as a Series with feature names; sort by absolute value; plot top 10
257. For Decision Tree: extract `feature_importances_` similarly



258. Identify the top 3 features most associated with higher churn probability

```

Top 3 Churn Drivers:
contract_Month-to-month    0.518854
internet_service_Fiber optic 0.164813
tenure                     0.164255
dtype: float64

```

259. Score all customers in the test set: `predict_proba()` to get churn probability

```

First 10 churn probabilities:
[0. 0.72139303 0.09430605 0.50503018 0. 0.36633663
 0.36633663 0.09623431 0.0162413 0.28953229]

```

260. Find the top 20 highest-risk customers in the test set — print their customer IDs and risk scores

```

Top 20 Highest-Risk Customers:
customer_i_d  churn_probability
3716-BDVDB    1.000
5542-TBBWB    1.000
8375-DKEBR    1.000
1820-TQVEV    1.000
0679-IDSTG    1.000
3750-CKVKH    1.000
5186-SAMNZ    1.000
9804-ICWBG    1.000
2245-ADZFJ    1.000
2867-UTMSS    0.825
5376-DEQCP    0.825
2202-CUYXZ    0.825
7181-BQYBV    0.825
4415-IJZTP    0.825
2254-DLXRI    0.825
8380-PEFPE    0.825
9248-OJYKK    0.825
6857-VWJDT    0.825
8821-XNHVZ    0.825
6372-RFVNS    0.825

```

261. Check: do the top-risk customers match the rule-based high-risk list from SQL6?
(partial overlap expected)
19 out of the 20 high risk customers identified can be found in the v_high_risk_customers view created in sql6
262. Write a 5-sentence business interpretation: "Customers most likely to churn are those who..." targeted at a retention manager
Customers most likely to churn are those on month-to-month contracts, customers using fiber-optic internet, and customers with shorter tenure. The strongest churn driver is the month-to-month contract, with a feature importance of 0.519, indicating that contract type plays a major role in the model's churn predictions. The ML model can identify high-risk customers beyond the SQL rule-based approach because it considers multiple customer characteristics simultaneously. The retention team should focus first on customers with the highest predicted churn probabilities. The top 20 customers identified by the ML model should be treated as a priority retention list, with the customers having predicted churn probabilities of 1.00 receiving the most immediate attention.

```

import pandas as pd
import joblib
import matplotlib.pyplot as plt

# Load CP4 CSV
df = pd.read_csv("output\customer_features_20260731_120356.csv")

# Load best model from ML2
tree_model = joblib.load("models/tree_churn.pkl")

# Same features used in ML2
numeric_cols = ["tenure", "monthly_charges", "total_charges", "service_count",
"high_charge_flag", "is_long_term_customer", "has_streaming_bundle", "auto_pay_flag"]
categorical_cols = ["contract", "internet_service"]

```

```

# Create X exactly as in ML2
X = df[numeric_cols + categorical_cols].copy()

X = pd.get_dummies(X, columns=categorical_cols, drop_first=False)

# Feature importances
importance = pd.Series(tree_model.feature_importances_, index=X.columns)
importance = importance.sort_values(ascending=False)

print("\nFeature Importances:")
print(importance)

top10 = importance.head(10).sort_values()

plt.figure(figsize=(8, 5))
top10.plot(kind="barh")

plt.title("Top 10 Features Driving Churn")
plt.xlabel("Feature Importance")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()

top3 = importance.head(3)

print("\nTop 3 Churn Drivers:")
print(top3)

from sklearn.model_selection import train_test_split

y = df["churn"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)

# 259. Predict churn probability
churn_prob = tree_model.predict_proba(X_test)[: , 1]

print("\nFirst 10 churn probabilities:")
print(churn_prob[:10])

```

```

test_customer_ids = df.loc[X_test.index, "customer_i_d"]
risk_df = pd.DataFrame({"customer_i_d": test_customer_ids, "churn_probability":
churn_prob})

risk_df = risk_df.sort_values(by="churn_probability", ascending=False)

top20 = risk_df.head(20)

print("\nTop 20 Highest-Risk Customers:")
print(top20.to_string(index=False))

```

LAB ML4 — Replace Stub API with Real Model

This section focused on integrating the trained machine learning model into the FastAPI application for real-time churn prediction. Initially, the model was trained using 14 features, but the prediction API accepted only four inputs (tenure, monthly_charges, contract_type, and service_count). To match the training schema, missing features were engineered where possible; however, internet_service, auto_pay_flag, and has_streaming_bundle could not be derived reliably from the available inputs and were temporarily assigned default values of 0. This significantly reduced prediction quality and produced inaccurate churn risk scores. To address this issue, these three features were added to the API request payload, while the remaining derived features continued to be engineered within the prediction pipeline. This ensured that the model received feature inputs that closely matched the training data, resulting in more accurate and reliable churn predictions

263. Create predict.py: load the best model from models/ at module level
264. Define preprocess_input(tenure, monthly_charges, contract_type, service_count) — creates a properly encoded single-row DataFrame matching the training feature columns
265. Define predict_churn(tenure, monthly_charges, contract_type, service_count) — calls preprocess_input, then model.predict() and predict_proba()
266. Return: {"risk_score": float, "prediction": "Likely to churn" / "Unlikely to churn", "confidence": float}
267. In main.py: import predict_churn from predict.py; replace the stub with a real call
268. Test via /docs: submit a month-to-month customer with short tenure → expect high risk

[Customer Search](#) | [Churn Summary](#) | [High Risk Customers](#) | [Churn Prediction](#)

Churn Prediction

Enter customer information to predict the likelihood of churn.

Tenure (months)

Monthly Charges

Contract Type

Service Count

Prediction Result

High Risk

Risk Score: 100.0%

Prediction: Likely to churn

Confidence Level: High

[Customer Search](#) | [Churn Summary](#) | [High Risk Customers](#) | [Churn Prediction](#)

Churn Prediction

Enter customer information to predict the likelihood of churn.

Tenure (months)

Monthly Charges

Contract Type

Service Count

Streaming Bundle

Auto Pay

Internet Service

Prediction Result

High Risk

Risk Score: 100.0%

Prediction: Likely to churn

Confidence Level: High

269. Test: submit a two-year contract customer with long tenure → expect low risk

[Customer Search](#) | [Churn Summary](#) | [High Risk Customers](#) | [Churn Prediction](#)

Churn Prediction

Enter customer information to predict the likelihood of churn.

Tenure (months)

Monthly Charges

Contract Type

Service Count

Prediction Result

Low Risk

Risk Score: 1.6%

Prediction: Unlikely to churn

Confidence Level: High

Churn Prediction

Enter customer information to predict the likelihood of churn.

Tenure (months)
Monthly Charges
Contract Type
Service Count
Streaming Bundle
Auto Pay
Internet Service

Prediction Result

Low Risk

Risk Score: 1.6%

Prediction: Unlikely to churn

Confidence Level: High

270. Full end-to-end: test from RE5 React form → API → model → coloured response in UI

```
import joblib
import pandas as pd

# Load model once when the application starts
model = joblib.load("models/tree_churn.pkl")

#Exact feature order used during training
FEATURE_COLUMNS = ["tenure", "monthly_charges", "total_charges", "service_count",
"high_charge_flag", "is_long_term_customer", "has_streaming_bundle", "auto_pay_flag",
"contract_Month-to-month", "contract_One year", "contract_Two year",
"internet_service_DSL", "internet_service_Fiber optic", "internet_service_No"]

def preprocess_input(tenure, monthly_charges, contract_type, service_count):
    # -----
    # Original numeric features
    # -----
    total_charges = monthly_charges * tenure
    high_charge_flag = 1 if monthly_charges > 70.35 else 0
    is_long_term_customer = 1 if tenure >= 24 else 0
    has_streaming_bundle = 0
    auto_pay_flag = 0
    contract_month_to_month = 0
    contract_one_year = 0
    contract_two_year = 0

    if contract_type == "Month-to-month":
        contract_month_to_month = 1
    elif contract_type == "One year":
```

```

        contract_one_year = 1
    elif contract_type == "Two year":
        contract_two_year = 1

    internet_service_dsl = 0
    internet_service_fiber = 0
    internet_service_no = 0

    data = {
        "tenure": tenure, "monthly_charges": monthly_charges, "total_charges":
total_charges,
        "service_count": service_count, "high_charge_flag": high_charge_flag,
        "is_long_term_customer": is_long_term_customer, "has_streaming_bundle":
has_streaming_bundle,
        "auto_pay_flag": auto_pay_flag, "contract_Month-to-month":
contract_month_to_month,
        "contract_One year": contract_one_year, "contract_Two year":
contract_two_year,
        "internet_service_DSL": internet_service_dsl, "internet_service_Fiber optic":
internet_service_fiber,
        "internet_service_No": internet_service_no
    }

    df = pd.DataFrame([data], columns=FEATURE_COLUMNS)
    return df

def predict_churn_ml(tenure, monthly_charges, contract_type, service_count):
    # Preprocess input
    features = preprocess_input(tenure, monthly_charges, contract_type,
service_count)

    # Model prediction
    prediction = model.predict(features)[0]

    # Probability of churn
    probabilities = model.predict_proba(features)[0]
    risk_score = probabilities[1]

    # Confidence = probability of predicted class
    confidence = max(probabilities)

    return {

```

```

        "risk_score": float(risk_score),
        "prediction": ("Likely to churn" if prediction == 1 else "Unlikely to
churn"),
        "confidence": float(confidence)
    }

```

```

def preprocess_input(tenure, monthly_charges, contract_type, service_count,
has_streaming_bundle, auto_pay_flag, internet_service):
    total_charges = monthly_charges * tenure
    high_charge_flag = 1 if monthly_charges > 70.35 else 0
    is_long_term_customer = 1 if tenure >= 24 else 0

    contract_month_to_month = 0
    contract_one_year = 0
    contract_two_year = 0

    if contract_type == "Month-to-month":
        contract_month_to_month = 1
    elif contract_type == "One year":
        contract_one_year = 1
    elif contract_type == "Two year":
        contract_two_year = 1

    internet_service_dsl = 0
    internet_service_fiber = 0
    internet_service_no = 0

    if internet_service == "DSL":
        internet_service_dsl = 1
    elif internet_service == "Fiber optic":
        internet_service_fiber = 1
    elif internet_service == "No":
        internet_service_no = 1

    data = {
        "tenure": tenure,
        "monthly_charges": monthly_charges,
        "total_charges": total_charges,
        "service_count": service_count,
        "high_charge_flag": high_charge_flag,
        "is_long_term_customer": is_long_term_customer,
        "has_streaming_bundle": has_streaming_bundle,

```

```

        "auto_pay_flag": auto_pay_flag,
        "contract_Month-to-month": contract_month_to_month,
        "contract_One year": contract_one_year,
        "contract_Two year": contract_two_year,
        "internet_service_DSL": internet_service_dsl,
        "internet_service_Fiber optic": internet_service_fiber,
        "internet_service_No": internet_service_no
    }
    df = pd.DataFrame([data], columns=FEATURE_COLUMNS)
    return df

def predict_churn_ml(tenure, monthly_charges, contract_type, service_count,
has_streaming_bundle, auto_pay_flag, internet_service):
    features = preprocess_input(tenure, monthly_charges, contract_type,
service_count, has_streaming_bundle, auto_pay_flag, internet_service)
    # Model prediction
    prediction = model.predict(features)[0]
    # Probability
    probabilities = model.predict_proba(features)[0]
    risk_score = probabilities[1]
    confidence = max(probabilities)
    return {
        "risk_score": float(risk_score),
        "prediction": ("Likely to churn" if prediction == 1 else "Unlikely to
churn"),
        "confidence": float(confidence)
    }

```

```

from ml4_predict import predict_churn_ml
@app.post(
    "/predict-churn",
    dependencies=[Depends(verify_api_key)]
)
def predict_churn(request: ChurnPredictionRequest):
    try:
        result = predict_churn_ml(
            request.tenure,
            request.monthly_charges,
            request.contract_type,
            request.service_count,
            request.has_streaming_bundle,
            request.auto_pay_flag,
            request.internet_service

```

```

    )
    return result
except Exception:
    logger.exception(
        "Error during churn prediction"
    )
    raise HTTPException(
        status_code=500,
        detail="Model prediction failed"
    )

```

LAB ML5 — Build a Batch Scoring Job

271. Create batch_score.py
272. Load all customers from customer_ml_features
273. Apply predict_churn() to every row using df.apply()
274. Extract risk_score, prediction, confidence into new columns
275. Add a scoring_date column: datetime.today().strftime("%Y-%m-%d")
276. Save to customer_risk_table (SQL table or CSV): customer_id, risk_score, prediction, confidence, scoring_date
277. Print a summary: count of "Likely to churn" vs "Unlikely to churn" — verify ~27% churn rate aligns with training distribution

```

Prediction Percentages:
prediction
Unlikely to churn    72.72
Likely to churn      27.28
Name: proportion, dtype: float64

```

278. Print top 10 highest risk_score customers — cross-check against SQL6 view (do the rule-based high-risk customers have high ML scores?)

```

Top 10 Highest-Risk Customers:
customer_id  risk_score  prediction  confidence
6993-OHLXR      1.0  Likely to churn      1.0
7206-GZCDC      1.0  Likely to churn      1.0
6457-GIRWB      1.0  Likely to churn      1.0
4501-EQDRN      1.0  Likely to churn      1.0
2245-ADZFJ      1.0  Likely to churn      1.0
5875-YPQFJ      1.0  Likely to churn      1.0
9804-ICWBG      1.0  Likely to churn      1.0
1062-LHZOD      1.0  Likely to churn      1.0
7660-HDPJV      1.0  Likely to churn      1.0
1084-UQCHV      1.0  Likely to churn      1.0

```

```
mysql> SELECT count(*)
-> FROM v_high_risk_customers
-> WHERE customer_id IN (
-> '6993-OHLXR', '7206-GZCDC', '6457-GIRWB', '4501-EQDRN', '2245-ADZFJ', '5875-YPQFJ', '9804-ICWBG', '1062-LHZOD', '7660
-HDPJV', '1084-UQCHV'
-> );
+-----+
| count(*) |
+-----+
|          9 |
+-----+
1 row in set (0.01 sec)
```

279. Discuss: how would you add this batch scoring as the 7th task in the DE7 Airflow DAG?

```
import pandas as pd
import joblib
from datetime import datetime
from sqlalchemy import create_engine

model = joblib.load("models/tree_churn.pkl")
df = pd.read_csv("customer_features1.csv")

print("Total customers:", len(df))

numeric_cols = ["tenure", "monthly_charges", "total_charges", "service_count",
"high_charge_flag", "is_long_term_customer", "has_streaming_bundle", "auto_pay_flag"]
categorical_cols = ["contract", "internet_service"]

X = df[numeric_cols + categorical_cols].copy()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=False)

probabilities = model.predict_proba(X)[: , 1]
predictions = model.predict(X)

# Convert prediction to business-friendly label
prediction_labels = ["Likely to churn" if prediction == 1 else "Unlikely to churn"
for prediction in predictions]

# Confidence = probability of predicted class
confidence = [probability if prediction == 1 else 1 - probability for probability,
prediction in zip(probabilities, predictions)]

scoring_date = datetime.today().strftime("%Y-%m-%d")

risk_df = pd.DataFrame({
    "customer_id": df["customer_id"],
```

```
    "risk_score": probabilities,
    "prediction": prediction_labels,
    "confidence": confidence,
    "scoring_date": scoring_date
}))

# Save CSV
# risk_df.to_csv("customer_risk_table.csv", index=False)
# print("\nCustomer risk table saved:")
# print("customer_risk_table.csv")

print("\nPrediction Summary:")
print(risk_df["prediction"].value_counts())

print("\nPrediction Percentages:")
print(risk_df["prediction"].value_counts(normalize=True).mul(100).round(2))

print("\nTop 10 Highest-Risk Customers:")

top10 = risk_df.sort_values(by="risk_score", ascending=False).head(10)

print(top10[["customer_id", "risk_score", "prediction",
"confidence"]].to_string(index=False))
```